Beside 백엔드 성능 보고서

측정일 2026-11-01 · 반복 v0 3회, v1 3회, v1p 3회, v2 3회, v2p 3회, v2q 3회 · 기준 v1

> ⚠ **다른 프로그램이 돌던 PC**: 스위트 직전 CPU 사용률 중앙 9 %, 최대 52 % (측정 방법의 '한계'). 버전 순서를 반복마다 돌려 한 버전에만 몰리지 않게 했고, 판정 규칙이 반복 사이 흔들림을 걸러 낸다.

버전

라벨	제목	커밋	설명
v0	이전 (10-14, #1 머지)	`e0e1c2b34` e0e1c2b34b1b94669d824fd0fd4afeea9fb800f1	origin/main. Unity 앱·AnimalLift 연동 직후. 업로드 전송·prod 프로파일 없음.
v1 (기준)	현재 (4b 배포 기반)	`af4a747b1` af4a747b1c36f2380d1ff69a523b5a9dbaca7503	main = deploy/vm-4b. prod 프로파일, /infer 업로드 전송, GPU 감시·오프라인 대기, Discord 알림, H2 백업.
v1p	현재 + 개선	`033f3f371` 033f3f3719745e3d4776c5ea6e4e2e7db536f027	perf/backend-report. v1 + P1(작업 목록 키셋 페이지, jobs(created_at, id) 인덱스).
v2	다음 버전 후보 (v1.1)	`6a3959f60` 6a3959f60380947ad2f430be24f5d50661a57c95	review/api-v1.1 의 rebase 전(백업 ref backup/review-api-v1.1-6a3959f). v1 + 관리 화면·대기 순번·취소·공유·평가·보관함·썸네일. 계약 회의 전.
v2p	다음 버전 + 개선	`ecc5fb5f9` ecc5fb5f9ba0bee443137ce8c59a5223174e7dc2	v2 를 v1p 위로 rebase + P4(썸네일을 202 뒤 백그라운드로) + P5(jobs·job_runs 인덱스, 대기 순번 범위 쿼리).
v2q	다음 버전 + 개선 2	`87e3c75ce` 87e3c75ce1be65a4a7eacd692b4ab115298a422e	review/api-v1.1 87e3c75. v2p 를 본선(perf/backend-report) 위로 rebase + 코드 리뷰 수정 + P6(썸네일 유무를 메모리에서) + P7(PENDING 대기 순번의 두 조회를 읽기 전용 트랜잭션 하나로).

핵심 지표

칸: p50 / p95 (ms) 또는 값, 아래 줄은 기준 대비 p50 변화율과 판정. 판정은 |Δ| ≥ 10% 이고 반복별 p50 범위가 기준과 겹치지 않을 때만 개선/악화, 나머지는 차이 없음(잡음). 처리량은 클수록 개선. 기준이 0 이면 비교 불가.

지표	v0	v1	v1p	v2	v2p	v2q
기동 시간 `bench.startupMs` · ms · 값	7,389 3회 −7.6% = 차이 없음	7,998 3회 기준	8,190 3회 +2.4% = 차이 없음	8,955 3회 +12.0% ! 악화	8,778 3회 +9.8% = 차이 없음	8,633 3회 +7.9% = 차이 없음
유휴 메모리 (RSS) `bench.idleRssMB` · MiB · 값	354.1 3회 −2.1% = 차이 없음	361.8 3회 기준	362.0 3회 +0.1% = 차이 없음	370.8 3회 +2.5% = 차이 없음	380.8 3회 +5.2% = 차이 없음	370.6 3회 +2.4% = 차이 없음
healthz 응답 `healthzMs` · ms · p50 / p95	0.44 / 2.74 n=1500 −10.2% = 차이 없음	0.49 / 2.03 n=1500 기준	0.47 / 2.31 n=1500 −4.1% = 차이 없음	0.46 / 2.92 n=1500 −6.1% = 차이 없음	0.47 / 2.69 n=1500 −5.1% = 차이 없음	0.46 / 2.57 n=1500 −6.1% = 차이 없음
작은 사진 업로드 `postMs` · ms · p50 / p95	11.9 / 39.1 n=300 −7.0% = 차이 없음	12.8 / 35.4 n=300 기준	12.4 / 34.7 n=300 −2.8% = 차이 없음	17.7 / 39.3 n=300 +38.8% ! 악화	16.6 / 55.6 n=300 +30.4% ! 악화	17.6 / 46.8 n=300 +38.2% ! 악화
큰 사진 4장 업로드 `postMs` · ms · p50 / p95	126 / 146 n=60 +1.5% = 차이 없음	124 / 148 n=60 기준	124 / 140 n=60 +0.2% = 차이 없음	341 / 368 n=60 +175.5% ! 악화	127 / 169 n=60 +2.2% = 차이 없음	129 / 157 n=60 +3.8% = 차이 없음
12 MP 사진 업로드 `postMs` · ms · p50 / p95	13.9 / 19.4 n=60 +0.2% = 차이 없음	13.9 / 16.7 n=60 기준	14.1 / 25.4 n=60 +1.8% = 차이 없음	64.4 / 82.4 n=60 +364.5% ! 악화	14.6 / 26.8 n=60 +5.5% = 차이 없음	14.3 / 19.9 n=60 +2.9% = 차이 없음
12 MP 동시 업로드 `postConcurrentMs` · ms · p50 / p95	46.8 / 116 n=60 −4.1% = 차이 없음	48.8 / 101 n=60 기준	53.6 / 131 n=60 +9.9% = 차이 없음	222 / 312 n=60 +354.9% ! 악화	64.9 / 121 n=60 +33.2% = 차이 없음	58.1 / 128 n=60 +19.1% ! 악화
큰 사진 업로드의 서버 CPU 점유 `bench.serverCpuMs` · ms · 값	3,609 3회 −1.7%	3,672 3회 기준	3,625 3회 −1.3%	4,375 3회 +19.1%	4,766 3회 +29.8%	4,875 3회 +32.8%
E2E 흐름 (mock) `flowMs` · ms · p50 / p95	22.4 / 32.3 n=150 +8.0% = 차이 없음	20.8 / 34.3 n=150 기준	24.4 / 34.2 n=150 +17.6% = 차이 없음	19.3 / 33.5 n=150 −7.3% = 차이 없음	31.6 / 48.5 n=150 +52.3% ! 악화	30.5 / 46.1 n=150 +46.9% = 차이 없음
동시 E2E 흐름 `flowMs` · ms · p50 / p95	186 / 431 n=480 −6.4% = 차이 없음	199 / 502 n=480 기준	191 / 477 n=480 −4.1% = 차이 없음	308 / 463 n=480 +55.1% ! 악화	282 / 641 n=480 +41.5% ! 악화	262 / 627 n=480 +31.8% ! 악화
동시 E2E 처리량 `bench.jobsPerSec` · 건/s · 값	67.2 3회 +1.8% = 차이 없음	66.0 3회 기준	64.1 3회 −2.9% = 차이 없음	50.6 3회 −23.4% = 차이 없음	43.9 3회 −33.5% ! 악화	49.0 3회 −25.8% ! 악화

지표	v0	v1	v1p	v2	v2p	v2q
폴링 부하 응답 `pollMs` · ms · p50 / p95	5.75 / 23.7 n=12000 −5.0% = 차이 없음	6.05 / 15.7 n=12000 기준	5.59 / 18.6 n=12000 −7.6% = 차이 없음	7.14 / 33.5 n=12000 +18.0% = 차이 없음	6.61 / 35.6 n=12000 +9.3% = 차이 없음	4.96 / 25.5 n=12000 −17.9% = 차이 없음
폴링 처리량 `bench.pollReqPerSec` · 요청/s · 값	1,740.0 3회 −2.9% = 차이 없음	1,792.4 3회 기준	1,837.4 3회 +2.5% = 차이 없음	1,365.9 3회 −23.8% ! 악화	1,326.7 3회 −26.0% ! 악화	1,637.9 3회 −8.6% = 차이 없음
목록 첫 페이지 (N=2000) `listFirstMs@2000` · ms · p50 / p95	10.9 / 19.6 n=300 +4.4% = 차이 없음	10.5 / 18.6 n=300 기준	3.59 / 4.90 n=300 −65.8% ✓ 개선	12.2 / 19.4 n=300 +16.3% ! 악화	5.47 / 7.90 n=300 −47.8% ✓ 개선	3.73 / 5.11 n=300 −64.4% ✓ 개선
목록 중간 페이지 (N=2000) `listDeepMs@2000` · ms · p50 / p95	20.1 / 30.1 n=150 +8.9% = 차이 없음	18.5 / 29.0 n=150 기준	11.1 / 17.8 n=150 −40.1% ✓ 개선	28.3 / 34.0 n=150 +53.1% ! 악화	22.7 / 32.8 n=150 +23.0% = 차이 없음	12.1 / 17.5 n=150 −34.7% ✓ 개선
단건 조회 (N=2000) `getMs@2000` · ms · p50 / p95	0.46 / 0.61 n=600 ±0.0% = 차이 없음	0.46 / 0.60 n=600 기준	0.45 / 0.59 n=600 −2.2% = 차이 없음	0.56 / 0.75 n=600 +21.7% ! 악화	0.65 / 0.84 n=600 +41.3% ! 악화	0.50 / 0.69 n=600 +8.7% = 차이 없음
업로드 (N=2000) `postMs@2000` · ms · p50 / p95	4.82 / 7.50 n=90 −1.6% = 차이 없음	4.90 / 7.13 n=90 기준	4.89 / 7.36 n=90 −0.1% = 차이 없음	9.35 / 11.9 n=90 +90.8% ! 악화	6.01 / 13.2 n=90 +22.8% ! 악화	6.01 / 12.4 n=90 +22.6% = 차이 없음
재기동 시간 `bench.restartStartupMs` · ms · 값	7,961 3회 +0.7% = 차이 없음	7,904 3회 기준	7,852 3회 −0.7% = 차이 없음	8,868 3회 +12.2% ! 악화	8,812 3회 +11.5% ! 악화	8,790 3회 +11.2% ! 악화
PENDING 조회 `pendingPollMs` · ms · p50 / p95	2.35 / 6.70 n=600 +20.5% ! 악화	1.96 / 5.95 n=600 기준	2.22 / 6.83 n=600 +13.6% = 차이 없음	4.34 / 13.7 n=600 +122.2% ! 악화	4.48 / 14.4 n=600 +129.2% ! 악화	3.40 / 11.9 n=600 +73.7% ! 악화
최대 메모리 (RSS) `bench.peakRssMB` · MiB · 값	506.8 3회 −1.4% = 차이 없음	513.9 3회 기준	450.4 3회 −12.4% ✓ 개선	517.0 3회 +0.6% = 차이 없음	513.9 3회 ±0.0% = 차이 없음	509.2 3회 −0.9% = 차이 없음
real · path 흐름 `flowMs` · ms · p50 / p95	75.4 / 97.2 n=60 −1.6% = 차이 없음	76.6 / 96.2 n=60 기준	76.2 / 93.1 n=60 −0.5% = 차이 없음	269 / 297 n=60 +250.9% ! 악화	80.8 / 112 n=60 +5.5% = 차이 없음	82.5 / 109 n=60 +7.8% = 차이 없음
real · upload 흐름 `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	306 / 343 n=60 기준	314 / 369 n=60 +2.6% = 차이 없음	311 / 359 n=60 +1.6% = 차이 없음	315 / 375 n=60 +3.0% = 차이 없음	325 / 372 n=60 +6.4% = 차이 없음
real,prod · upload 흐름 `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	313 / 347 n=60 기준	310 / 347 n=60 −0.9% = 차이 없음	311 / 358 n=60 −0.4% = 차이 없음	311 / 368 n=60 −0.4% = 차이 없음	322 / 401 n=60 +3.0% = 차이 없음
작업 사이 빈 시간 (prod) `queueGapMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	14.0 / 26.3 n=12 기준	14.8 / 18.8 n=12 +5.4% = 차이 없음	15.1 / 24.6 n=12 +7.4% = 차이 없음	19.5 / 33.0 n=12 +39.0% = 차이 없음	27.6 / 40.0 n=12 +96.6% ! 악화

쌍 비교

versions 파일의 `pairs`: 열마다 앞 버전을 기준으로 뒤 버전을 비교한다. 칸: 앞 → 뒤의 p50(값 지표는 반복별 값의 중앙값), 아래 줄은 변화율과 판정(위 표와 같은 규칙).

지표	이전 → 현재 v0 → v1	현재의 개선 (P1) v1 → v1p	다음 버전 기능의 비용 v1 → v2	다음 버전의 개선 (P1·P4·P5) v2 → v2p	다음 버전의 개선 2 (리뷰 수정·P6·P7) v2p → v2q
기동 시간 `bench.startupMs` · ms	7,389 → 7,998 +8.2% = 차이 없음	7,998 → 8,190 +2.4% = 차이 없음	7,998 → 8,955 +12.0% ! 악화	8,955 → 8,778 −2.0% = 차이 없음	8,778 → 8,633 −1.6% = 차이 없음
유휴 메모리 (RSS) `bench.idleRssMB` · MiB	354.1 → 361.8 +2.2% = 차이 없음	361.8 → 362.0 +0.1% = 차이 없음	361.8 → 370.8 +2.5% = 차이 없음	370.8 → 380.8 +2.7% = 차이 없음	380.8 → 370.6 −2.7% = 차이 없음
healthz 응답 `healthzMs` · ms	0.44 → 0.49 +11.4% = 차이 없음	0.49 → 0.47 −4.1% = 차이 없음	0.49 → 0.46 −6.1% = 차이 없음	0.46 → 0.47 +1.1% = 차이 없음	0.47 → 0.46 −1.1% = 차이 없음
작은 사진 업로드 `postMs` · ms	11.9 → 12.8 +7.5% = 차이 없음	12.8 → 12.4 −2.8% = 차이 없음	12.8 → 17.7 +38.8% ! 악화	17.7 → 16.6 −6.1% = 차이 없음	16.6 → 17.6 +6.0% = 차이 없음
큰 사진 4장 업로드 `postMs` · ms	126 → 124 −1.5% = 차이 없음	124 → 124 +0.2% = 차이 없음	124 → 341 +175.5% ! 악화	341 → 127 −62.9% ✓ 개선	127 → 129 +1.6% = 차이 없음
12 MP 사진 업로드 `postMs` · ms	13.9 → 13.9 −0.2% = 차이 없음	13.9 → 14.1 +1.8% = 차이 없음	13.9 → 64.4 +364.5% ! 악화	64.4 → 14.6 −77.3% ✓ 개선	14.6 → 14.3 −2.5% = 차이 없음

지표	이전 → 현재 v0 → v1	현재의 개선 (P1) v1 → v1p	다음 버전 기능의 비용 v1 → v2	다음 버전의 개선 (P1·P4·P5) v2 → v2p	다음 버전의 개선 2 (리뷰 수정·P6·P7) v2p → v2q
12 MP 동시 업로드 `postConcurrentMs` · ms	46.8 → 48.8 +4.3% = 차이 없음	48.8 → 53.6 +9.9% = 차이 없음	48.8 → 222 +354.9% ! 악화	222 → 64.9 −70.7% ✓ 개선	64.9 → 58.1 −10.5% = 차이 없음
큰 사진 업로드의 서버 CPU 점유 `bench.serverCpuMs` · ms	3,609 → 3,672 +1.7%	3,672 → 3,625 −1.3%	3,672 → 4,375 +19.1%	4,375 → 4,766 +8.9%	4,766 → 4,875 +2.3%
E2E 흐름 (mock) `flowMs` · ms	22.4 → 20.8 −7.4% = 차이 없음	20.8 → 24.4 +17.6% = 차이 없음	20.8 → 19.3 −7.3% = 차이 없음	19.3 → 31.6 +64.3% ! 악화	31.6 → 30.5 −3.5% = 차이 없음
동시 E2E 흐름 `flowMs` · ms	186 → 199 +6.8% = 차이 없음	199 → 191 −4.1% = 차이 없음	199 → 308 +55.1% ! 악화	308 → 282 −8.7% = 차이 없음	282 → 262 −6.9% = 차이 없음
동시 E2E 처리량 `bench.jobsPerSec` · 건/s	67.2 → 66.0 −1.8% = 차이 없음	66.0 → 64.1 −2.9% = 차이 없음	66.0 → 50.6 −23.4% = 차이 없음	50.6 → 43.9 −13.2% = 차이 없음	43.9 → 49.0 +11.7% = 차이 없음
폴링 부하 응답 `pollMs` · ms	5.75 → 6.05 +5.2% = 차이 없음	6.05 → 5.59 −7.6% = 차이 없음	6.05 → 7.14 +18.0% = 차이 없음	7.14 → 6.61 −7.4% = 차이 없음	6.61 → 4.96 −24.9% = 차이 없음
폴링 처리량 `bench.pollReqPerSec` · 요청/s	1,740.0 → 1,792.4 +3.0% = 차이 없음	1,792.4 → 1,837.4 +2.5% = 차이 없음	1,792.4 → 1,365.9 −23.8% ! 악화	1,365.9 → 1,326.7 −2.9% = 차이 없음	1,326.7 → 1,637.9 +23.5% ✓ 개선
목록 첫 페이지 (N=2000) `listFirstMs@2000` · ms	10.9 → 10.5 −4.2% = 차이 없음	10.5 → 3.59 −65.8% ✓ 개선	10.5 → 12.2 +16.3% ! 악화	12.2 → 5.47 −55.1% ✓ 개선	5.47 → 3.73 −31.8% ✓ 개선
목록 중간 페이지 (N=2000) `listDeepMs@2000` · ms	20.1 → 18.5 −8.2% = 차이 없음	18.5 → 11.1 −40.1% ✓ 개선	18.5 → 28.3 +53.1% ! 악화	28.3 → 22.7 −19.7% ✓ 개선	22.7 → 12.1 −46.9% ✓ 개선
단건 조회 (N=2000) `getMs@2000` · ms	0.46 → 0.46 ±0.0% = 차이 없음	0.46 → 0.45 −2.2% = 차이 없음	0.46 → 0.56 +21.7% ! 악화	0.56 → 0.65 +16.1% = 차이 없음	0.65 → 0.50 −23.1% = 차이 없음
업로드 (N=2000) `postMs@2000` · ms	4.82 → 4.90 +1.7% = 차이 없음	4.90 → 4.89 −0.1% = 차이 없음	4.90 → 9.35 +90.8% ! 악화	9.35 → 6.01 −35.7% ✓ 개선	6.01 → 6.01 −0.1% = 차이 없음
재기동 시간 `bench.restartStartupMs` · ms	7,961 → 7,904 −0.7% = 차이 없음	7,904 → 7,852 −0.7% = 차이 없음	7,904 → 8,868 +12.2% ! 악화	8,868 → 8,812 −0.6% = 차이 없음	8,812 → 8,790 −0.2% = 차이 없음
PENDING 조회 `pendingPollMs` · ms	2.35 → 1.96 −17.0% ✓ 개선	1.96 → 2.22 +13.6% = 차이 없음	1.96 → 4.34 +122.2% ! 악화	4.34 → 4.48 +3.1% = 차이 없음	4.48 → 3.40 −24.2% ✓ 개선
최대 메모리 (RSS) `bench.peakRssMB` · MiB	506.8 → 513.9 +1.4% = 차이 없음	513.9 → 450.4 −12.4% ✓ 개선	513.9 → 517.0 +0.6% = 차이 없음	517.0 → 513.9 −0.6% = 차이 없음	513.9 → 509.2 −0.9% = 차이 없음
real · path 흐름 `flowMs` · ms	75.4 → 76.6 +1.6% = 차이 없음	76.6 → 76.2 −0.5% = 차이 없음	76.6 → 269 +250.9% ! 악화	269 → 80.8 −69.9% ✓ 개선	80.8 → 82.5 +2.1% = 차이 없음
real · upload 흐름 `flowMs` · ms	–	306 → 314 +2.6% = 차이 없음	306 → 311 +1.6% = 차이 없음	311 → 315 +1.4% = 차이 없음	315 → 325 +3.2% = 차이 없음
real,prod · upload 흐름 `flowMs` · ms	–	313 → 310 −0.9% = 차이 없음	313 → 311 −0.4% = 차이 없음	311 → 311 ±0.0% = 차이 없음	311 → 322 +3.5% = 차이 없음
작업 사이 빈 시간 (prod) `queueGapMs` · ms	–	14.0 → 14.8 +5.4% = 차이 없음	14.0 → 15.1 +7.4% = 차이 없음	15.1 → 19.5 +29.5% ! 악화	19.5 → 27.6 +41.4% ! 악화

측정 방법

환경

항목	값
OS	Windows-11-10.0.26300-SP0
CPU	AMD Ryzen 5 5600X 6-Core Processor
논리 CPU	12
메모리 (GB)	31.9
Java	openjdk version "17.0.20.1" 2026-08-18 LTS
Python (부하 생성기)	3.14.0
JVM 옵션	-Xms256m -Xmx1536m -XX:ActiveProcessorCount=2 -XX:+UseG1GC
서버 CPU 고정	10, 11 (마지막 논리 CPU, 물리 코어 1개)
전원	source ac, battery none, plan 고성능 (8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c)

서버 조건

- JVM: `-Xms256m -Xmx1536m -XX:ActiveProcessorCount=2 -XX:+UseG1GC`, 버전마다 `bootJar` 로 만든 jar 를 `java -jar` 로 실행.
- 서버 프로세스는 마지막 논리 CPU 10, 11 에 고정했다(물리 코어 1개를 SMT 로 나눈 논리 CPU 2개라 계산 자원은 코어 1개 분량이다). JVM 에는 `-XX:ActiveProcessorCount=2` 로 CPU 2개라고 알렸다. 부하 생성기(Python)와 추론 스텁은 고정하지 않는다.
- 서버 A·B: 프로파일 `mock`, 지연 0 (`mock.worker.pending-ms=0`, `processing-ms=0`), 실패 트리거 끔, 결과 GLB 는 합성 GLB(1,500,000 바이트). 서버 B 는 A 의 저장소로 다시 띄우고 `pending-ms=600000` 으로 작업을 PENDING 에 둔다.
- 서버 C: 프로파일 `real` 또는 `real,prod`, 추론 서비스 스텁(`generation/inference_service/app.py`, 패스스루: 모델 없이 같은 GLB 를 즉시 돌려줌), 전송 path / upload, 토큰 없음.
- 저장소(H2 파일 DB, 업로드, 결과, events.jsonl)는 작업 폴더 `C:\Users\minwo\AppData\Local\Temp\beside-perf` 의 실행 폴더 아래에 서버 A 와 서버 C(real 시나리오마다)를 띄울 때마다 새로 만든다. 서버 B 는 A 의 저장소를 그대로 다시 연다(재기동 시나리오). 모든 `storage.*` 경로는 절대 경로로 지정.
- 클라이언트: Python `http.client`, 127.0.0.1 루프백, 스레드마다 keep-alive 연결. 시간은 요청 쓰기 직전부터 응답 본문을 다 읽을 때까지(`perf_counter_ns`), 연결 수립은 제외 — E2E 흐름(`flowMs`·`completeMs`)도 흐름 안에서 다시 연결한 시간(Tomcat 은 100번째 응답에 연결을 닫는다)을 뺀다. POST 마다 새 `Idempotency-Key`.
- 반복마다 버전 순서를 돌린다(1회차 v0,v1,v2 → 2회차 v1,v2,v0 → ...). 워밍업 요청은 기록하지 않는다.
- 서버가 쉴 때까지 기다린다: 서버 프로세스가 한 창 동안 CPU 를 한 코어의 5 % 이하로 쓰면 쉬는 것으로 본다. 서버 A 의 시나리오가 끝날 때마다 1초 창(최대 120초), 큰 사진·12 MP 업로드와 real 흐름은 하나마다 시작 전에 0.5초 창(최대 30초), 12 MP 동시 업로드·scale 의 측정 전과 PENDING 작업을 만든 뒤에도 기다린다. 응답 뒤에 남은 일(백그라운드 썸네일 큐, GC, JIT)이 다음 측정에 섞이지 않게 하려는 것이다(실사용에서 업로드는 몇 분 간격). 쉬는 것을 확인한 창 하나를 뺀 대기는 `bench.settleAfterMs`·`bench.settleTotalMs`, 시나리오 시작부터 대기 끝까지 서버가 쓴 CPU 시간은 `bench.serverCpuMs` 다. CPU 시간은 Windows 에서 스케줄러 틱(약 15.6 ms) 단위이고 고정한 두 논리 CPU 가 한 물리 코어를 나눠 쓰므로 반복 사이 흔들림이 커서 변화율만 보이고 판정하지 않는다.
- 프로세스 우선순위: 서버·추론 스텁·부하 생성기를 '보통 이상'(Windows ABOVE_NORMAL)으로 띄웠다. 같은 PC 의 다른 프로그램이 서버를 고정한 CPU 를 차지해 서버가 몇 초씩 멈추는 일을 줄이려는 것이다(클라우드 VM 에는 다른 프로그램이 없다).

시나리오

이름	무엇을	횟수
서버 기동 `startup`	프로세스 시작(Popen) → 첫 healthz 200, Spring 'Started' 로그 시간, 그 3초 뒤 RSS. 서버 A, 빈 저장소.	서버당 1회
헬스 체크 `healthz`	GET /api/v1/healthz 순차, keep-alive 연결 하나.	워밍업 100 + 측정 500
작은 사진 업로드 `upload_small`	64×64 PNG(약 12 KB) 1장 POST /api/v1/jobs 순차.	워밍업 10 + 측정 100
큰 사진 업로드 `upload_large`	약 4.5 MB 노이즈 PNG 4장(요청 하나에 약 18 MB) POST 순차.	워밍업 2 + 측정 20, 요청당 4장
12 MP 사진 업로드 `upload_12mp`	4000×3000 그라디언트 PNG(약 1.1 MB, 디코딩 부담) 1장 POST 순차, 이어서 4스레드 동시.	워밍업 2 + 측정 20, 동시 4스레드 × 5
E2E 흐름 (mock) `e2e_mock`	POST → 10 ms 간격 폴링 → COMPLETED → GLB 다운로드(크기 확인), 순차.	워밍업 5 + 측정 50, 폴링 10 ms

이름	무엇을	횟수
동시 E2E 흐름 `concurrent_e2e`	16스레드가 각자 E2E 흐름을 반복, 폴링 간격 50 ms.	16스레드 × 10흐름, 폴링 50 ms
폴링 부하 `poll_load`	COMPLETED 작업 하나를 4프로세스 × 4스레드가 GET /api/v1/jobs/{id} 로 반복 조회.	4프로세스 × 4스레드 × 250 (스레드마다 연결용 워밍업 5)
작업 수에 따른 변화 `scale`	작업을 N 개까지 쌓은 뒤(8스레드 업로드) 목록 첫 페이지·중간 페이지, 단건 조회, 업로드.	N = 200, 1000, 2000; 목록 첫 100, 중간 50, 조회 200, 업로드 30 (각 워밍업 5, 업로드 제외)
자원 `resources`	mock 시나리오를 모두 마친 서버 A 의 최대 RSS(작업 집합 최대치)와 jar 크기.	1회
재기동·PENDING 폴링 `pending_poll`	서버 A 의 저장소로 재기동(서버 B, 작업이 PENDING 에 머묾): 기동 시간, PENDING 작업 조회.	PENDING 작업 5개, 워밍업 20 + 측정 200
real · path 전송 `real_path`	real 프로파일 + 추론 스텁(패스스루, 모델 없음), 사진 경로 전달. 4.5 MB PNG 1장 E2E 순차, 이어서 5건 동시 접수.	워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건
real · upload 전송 `real_upload`	real_path 와 같고 /infer/upload 로 사진을 올리고 GLB 를 내려받는다.	워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건
real,prod · upload 전송 `real_upload_prod`	real,prod 프로파일: 사진 시그니처 검사, 작업마다 추론 서비스 healthz 확인. upload 전송.	워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건

통계와 판정

- p50·p95 등 백분위수는 반복을 합친 표본에서 선형 보간(numpy 'linear', Hyndman–Fan type 7)으로 구한다. 원시 표본은 0.01 ms 단위로 저장한다.
- Δ% = (후보 p50 − 기준 p50) ÷ 기준 p50 × 100. 값 지표(기동 시간, RSS, 처리량)는 반복별 값의 중앙값끼리 비교한다.
- 판정: |Δ%| ≥ 10 이고 반복별 p50(값 지표는 반복별 값)의 [최소, 최대] 범위가 기준의 범위와 겹치지 않을 때만 **개선** 또는 **악화**. 그 밖에는 **차이 없음**(잡음으로 본다). 시간은 작을수록, 처리량은 클수록 개선. 기준이 0 이고 후보가 0 이 아니면 변화율이 없으므로 **비교 불가**.
- 오류 수(`bench.errors`)는 측정값이 아니라 사건 수라 반복을 **합한** 값을 보이고, 0 이 아니면 빨간색이며 판정하지 않는다. 실패한 요청이 하나라도 있으면 그 시나리오는 '오류 포함'으로 표시되고, 표본(p50·p95)은 성공한 요청만으로 계산된다.

이름 규칙

- `bench.*` 값과 클라이언트 표본 이름(`postMs`, `flowMs`, `pollMs`, `assetMs` 등)은 이 벤치마크 전용이며 docs/METRICS.md 에 등록된 이름이 아니다. 앱의 `uploadMs`(METRICS §3)와 섞지 않는다.
- 서버 값 `uploadMs`, `queuedMs`, `processingMs`, `totalMs` (표의 '· events' 행과 real 시나리오의 `server*` 표본)은 서버 events.jsonl 의 METRICS §1 정의 그대로다. real 시나리오는 스텁 패스스루라 `passthrough=true` 이고 모델 측정이 아니다.

한계

- 로컬 루프백(127.0.0.1)에서 잰 값이다. 클라우드 VM, Tailscale, 실제 GPU 추론 경로가 아니다.
- real 시나리오는 모델 없는 스텁이라 Spring 쪽 오버헤드(전송·저장·대기열)만 보인다.
- 부하 생성기는 Python(GIL, http.client)이라 동시 시나리오에는 클라이언트 자신의 지연이 섞일 수 있다.
- 서버와 클라이언트가 같은 PC 에서 돈다. 서버는 논리 CPU 10, 11(2개, 물리 코어 1개)에 고정했지만 부하 생성기와 추론 스텁은 고정하지 않았고, 디스크·메모리 대역은 공유한다. 백신 실시간 검사 등 OS 활동도 포함된다.
- 다른 프로그램의 부하: 스위트마다 시작 직전 2초 동안(이 도구의 서버는 꺼진 상태) 잰 PC 전체 CPU 사용률이 최소 5 %, 중앙 9 %, 최대 52 % 였다(스위트 18개). 서버를 고정한 CPU 도 다른 프로그램이 쓸 수 있어 이만큼 잡음이 섞인다.
- 갓 쓴 파일의 첫 열기가 느리다: 4.5 MB 사진 사본을 쓰자마자 열어 읽는 데 206 ms, 다시 읽으면 1.4 ms 였다(실시간 백신 검사로 보인다). 업로드 직후 사진을 여는 서버 코드(요청 안에서 만드는 썸네일, 추론 서비스로의 업로드 전송)는 이 대기를 포함하고, 그런 검사가 없는 Linux VM 에서는 이만큼 빠진다.
- 서버 쪽 시간(events.jsonl)은 ms 단위 정수(버림)라 1 ms 보다 작은 차이는 보이지 않는다 (이 측정의 서버: openjdk version "17.0.20.1" 2026-08-18 LTS · Windows-11-10.0.26300-SP0).

시나리오별 결과

표: p50 / p95 와 표본 수 n(값 지표는 반복의 중앙값), 아래 줄은 기준 대비 Δ% 와 판정. 그림: 같은 값을 막대로.

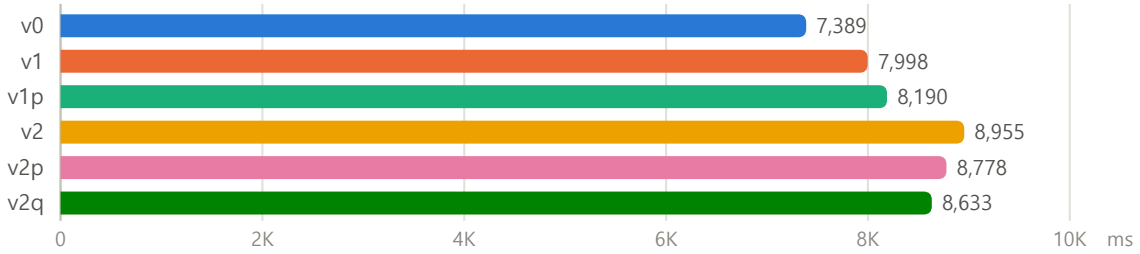
1. 서버 기동 `startup`

프로세스 시작(Popen) → 첫 healthz 200, Spring 'Started' 로그 시간, 그 3초 뒤 RSS. 서버 A, 빈 저장소. 서버당 1회.

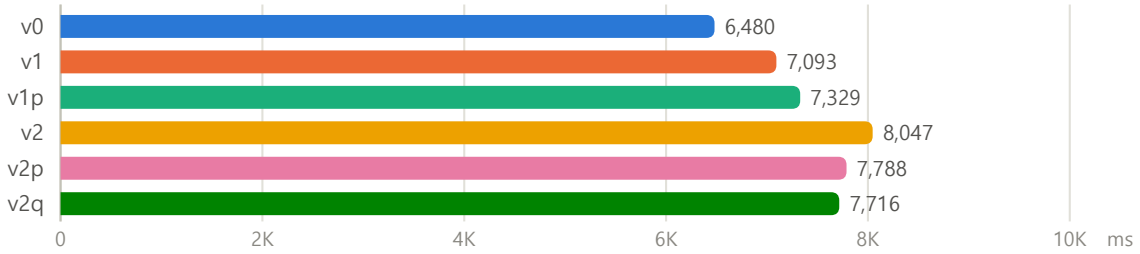
지표	v0	v1	v1p	v2	v2p	v2q
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	7,389 3회 −7.6% = 차이 없음	7,998 3회 기준	8,190 3회 +2.4% = 차이 없음	8,955 3회 +12.0% ! 악화	8,778 3회 +9.8% = 차이 없음	8,633 3회 +7.9% = 차이 없음
Spring 'Started' 로그 시간 `bench.springStartedMs` · ms · 값	6,480 3회 −8.6% = 차이 없음	7,093 3회 기준	7,329 3회 +3.3% = 차이 없음	8,047 3회 +13.4% ! 악화	7,788 3회 +9.8% = 차이 없음	7,716 3회 +8.8% = 차이 없음
유휴 RSS (healthz 3초 뒤) `bench.idleRssMB` · MiB · 값	354.1 3회 −2.1% = 차이 없음	361.8 3회 기준	362.0 3회 +0.1% = 차이 없음	370.8 3회 +2.5% = 차이 없음	380.8 3회 +5.2% = 차이 없음	370.6 3회 +2.4% = 차이 없음

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선 · v2q 다음 버전 + 개선 2

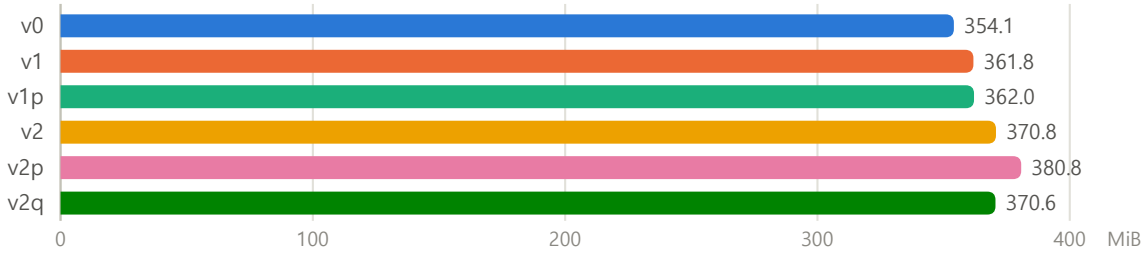
기동 (Popen→healthz 200) · bench.startupMs



Spring 'Started' 로그 시간 · bench.springStartedMs



유휴 RSS (healthz 3초 뒤) · bench.idleRssMB



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

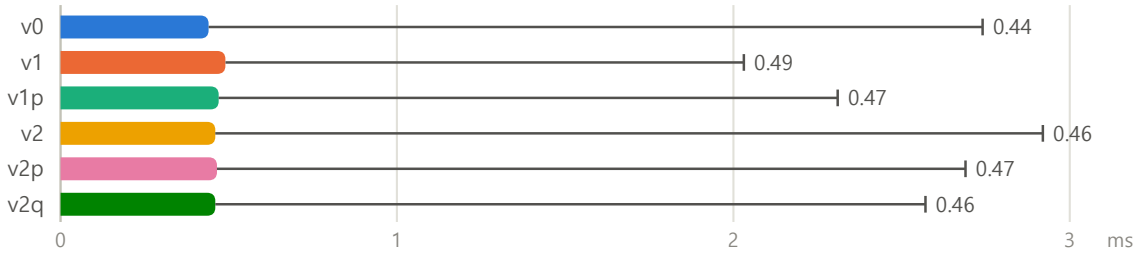
2. 헬스 체크 `healthz`

GET /api/v1/healthz 순차, keep-alive 연결 하나. 워밍업 100 + 측정 500.

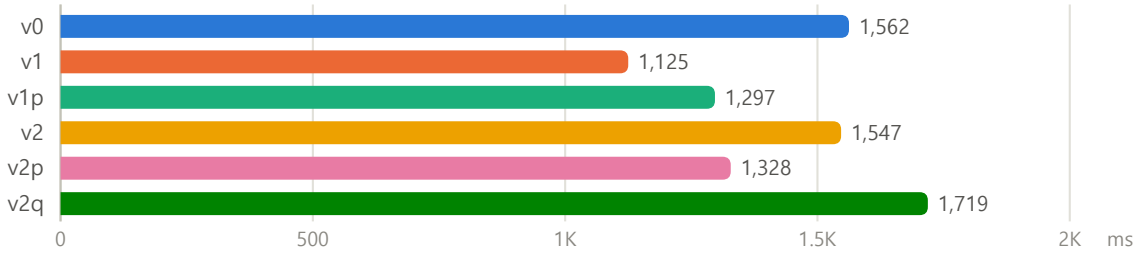
지표	v0	v1	v1p	v2	v2p	v2q
응답 시간 `healthzMs` · ms · p50 / p95	0.44 / 2.74 n=1500 −10.2% = 차이 없음	0.49 / 2.03 n=1500 기준	0.47 / 2.31 n=1500 −4.1% = 차이 없음	0.46 / 2.92 n=1500 −6.1% = 차이 없음	0.47 / 2.69 n=1500 −5.1% = 차이 없음	0.46 / 2.57 n=1500 −6.1% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	1,562 3회 +38.9%	1,125 3회 기준	1,297 3회 +15.3%	1,547 3회 +37.5%	1,328 3회 +18.1%	1,719 3회 +52.8%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	1,001 3회	0.36 3회	0.55 3회	1,000 3회	1,001 3회	1,001 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선
v2q 다음 버전 + 개선 2

응답 시간 · healthzMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

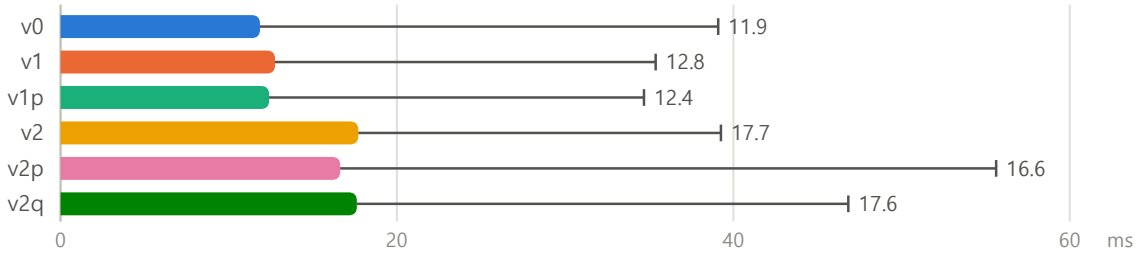
3. 작은 사진 업로드 `upload_small`

64×64 PNG(약 12 KB) 1장 POST /api/v1/jobs 순차. 워밍업 10 + 측정 100.

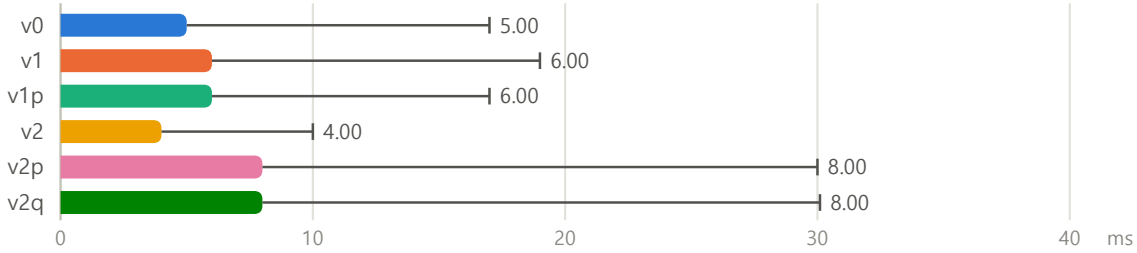
지표	v0	v1	v1p	v2	v2p	v2q
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	11.9 / 39.1 n=300 −7.0% = 차이 없음	12.8 / 35.4 n=300 기준	12.4 / 34.7 n=300 −2.8% = 차이 없음	17.7 / 39.3 n=300 +38.8% ! 악화	16.6 / 55.6 n=300 +30.4% ! 악화	17.6 / 46.8 n=300 +38.2% ! 악화
서버 uploadMs · events `postMs → uploadMs` · ms · p50 / p95	5.00 / 17.0 n=300 −16.7% = 차이 없음	6.00 / 19.0 n=300 기준	6.00 / 17.0 n=300 ±0.0% = 차이 없음	4.00 / 10.0 n=300 −33.3% ✓ 개선	8.00 / 30.0 n=300 +33.3% ! 악화	8.00 / 30.1 n=300 +33.3% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,188 3회 +5.9%	3,953 3회 기준	3,984 3회 +0.8%	5,141 3회 +30.0%	5,203 3회 +31.6%	5,109 3회 +29.2%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	1,000 3회	1,000 3회	1,000 3회	1,000 3회	1,001 3회	1,001 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선
v2q 다음 버전 + 개선 2

업로드 요청 (POST→202) · postMs



서버 uploadMs · events · postMs → uploadMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

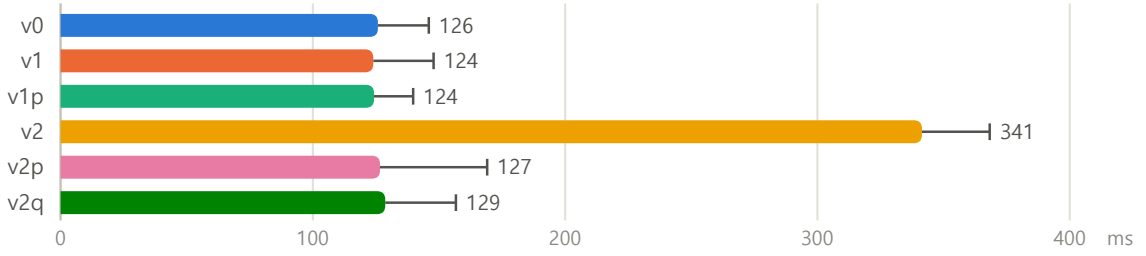
4. 큰 사진 업로드 `upload_large`

약 4.5 MB 노이즈 PNG 4장(요청 하나에 약 18 MB) POST 순차. 워밍업 2 + 측정 20, 요청당 4장.

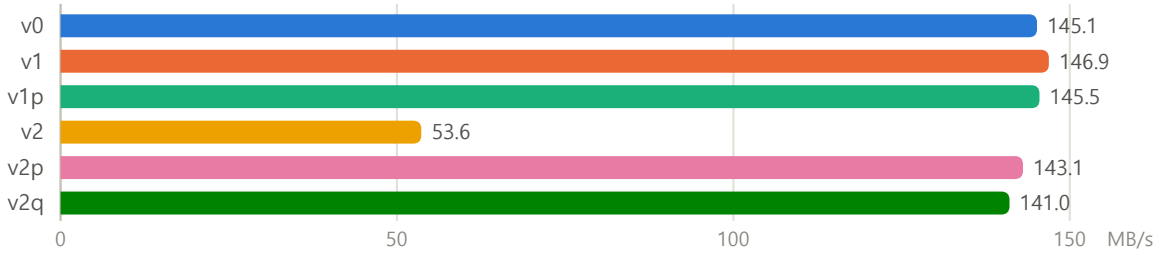
지표	v0	v1	v1p	v2	v2p	v2q
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	126 / 146 n=60 +1.5% = 차이 없음	124 / 148 n=60 기준	124 / 140 n=60 +0.2% = 차이 없음	341 / 368 n=60 +175.5% ! 악화	127 / 169 n=60 +2.2% = 차이 없음	129 / 157 n=60 +3.8% = 차이 없음
업로드 처리량 (요청 바이트 ÷ p50) `bench.uploadMBps` · MB/s · 값	145.1 3회 −1.2% = 차이 없음	146.9 3회 기준	145.5 3회 −0.9% = 차이 없음	53.6 3회 −63.5% ! 악화	143.1 3회 −2.6% = 차이 없음	141.0 3회 −4.0% = 차이 없음
서버 uploadMs · events `postMs → uploadMs` · ms · p50 / p95	118 / 137 n=60 +1.3% = 차이 없음	116 / 136 n=60 기준	117 / 131 n=60 +0.4% = 차이 없음	115 / 134 n=60 −1.3% = 차이 없음	118 / 148 n=60 +1.3% = 차이 없음	120 / 147 n=60 +3.0% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	3,609 3회 −1.7%	3,672 3회 기준	3,625 3회 −1.3%	4,375 3회 +19.1%	4,766 3회 +29.8%	4,875 3회 +32.8%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.45 3회	0.65 3회	0.35 3회	0.64 3회	1,001 3회	1.03 3회
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	4,009 3회	2,510 3회	3,510 3회	509 3회	8,012 3회	8,014 3회

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선 · v2q 다음 버전 + 개선 2

업로드 요청 (POST→202) · postMs



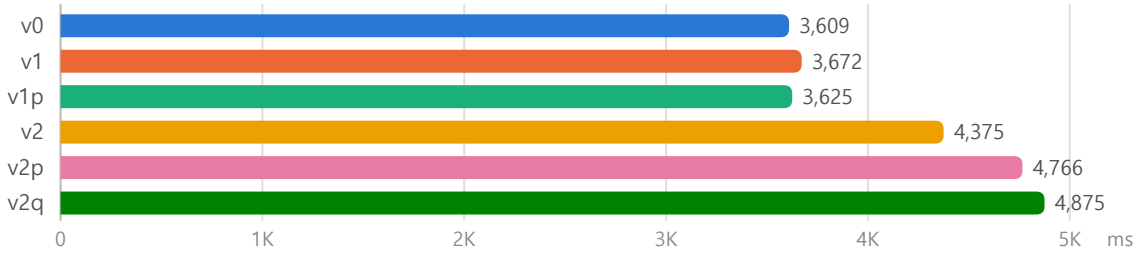
업로드 처리량 (요청 바이트 ÷ p50) · bench.uploadMBps



서버 uploadMs · events · postMs → uploadMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

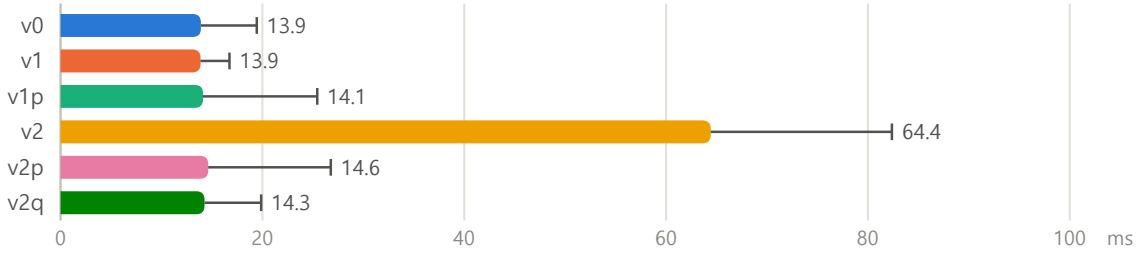
5. 12 MP 사진 업로드 `upload_12mp`

4000×3000 그라디언트 PNG(약 1.1 MB, 디코딩 부담) 1장 POST 순차, 이어서 4스레드 동시. 워밍업 2 + 측정 20, 동시 4스레드 × 5.

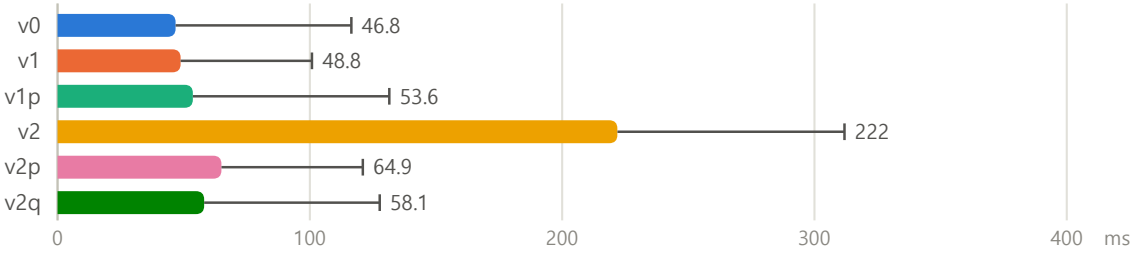
지표	v0	v1	v1p	v2	v2p	v2q
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	13.9 / 19.4 n=60 +0.2% = 차이 없음	13.9 / 16.7 n=60 기준	14.1 / 25.4 n=60 +1.8% = 차이 없음	64.4 / 82.4 n=60 +364.5% ! 악화	14.6 / 26.8 n=60 +5.5% = 차이 없음	14.3 / 19.9 n=60 +2.9% = 차이 없음
동시 업로드 (4스레드) `postConcurrentMs` · ms · p50 / p95	46.8 / 116 n=60 −4.1% = 차이 없음	48.8 / 101 n=60 기준	53.6 / 131 n=60 +9.9% = 차이 없음	222 / 312 n=60 +354.9% ! 악화	64.9 / 121 n=60 +33.2% = 차이 없음	58.1 / 128 n=60 +19.1% ! 악화
서버 uploadMs · events `postMs → uploadMs` · ms · p50 / p95	10.0 / 12.1 n=60 ±0.0% = 차이 없음	10.0 / 12.0 n=60 기준	10.0 / 13.1 n=60 ±0.0% = 차이 없음	10.0 / 13.2 n=60 ±0.0% = 차이 없음	10.0 / 13.4 n=60 ±0.0% = 차이 없음	10.0 / 12.1 n=60 ±0.0% = 차이 없음
서버 uploadMs · events `postConcurrentMs → uploadMs` · ms · p50 / p95	38.0 / 92.6 n=60 +4.1% = 차이 없음	36.5 / 98.2 n=60 기준	41.0 / 87.7 n=60 +12.3% = 차이 없음	11.5 / 51.5 n=60 −68.5% ✓ 개선	40.5 / 98.5 n=60 +11.0% = 차이 없음	49.5 / 124 n=60 +35.6% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	1,406 3회 +15.4%	1,219 3회 기준	1,719 3회 +41.0%	3,641 3회 +198.7%	3,609 3회 +196.2%	3,531 3회 +189.7%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	1,001 3회	0.17 3회	0.58 3회	0.27 3회	1,001 3회	1,001 3회
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	3,011 3회	2,008 3회	4,511 3회	2,008 3회	10,017 3회	11,018 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

업로드 요청 (POST→202) · postMs



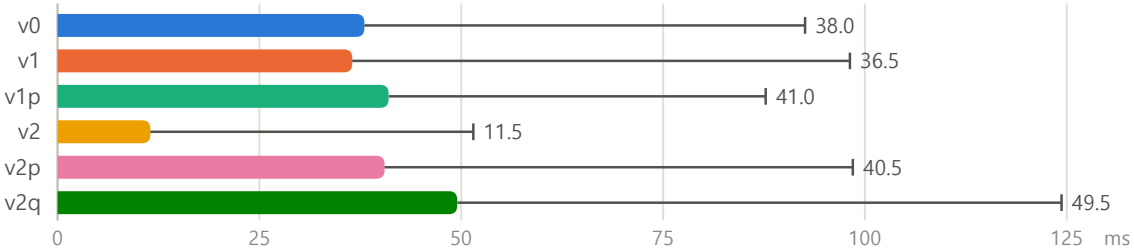
동시 업로드 (4스레드) · postConcurrentMs



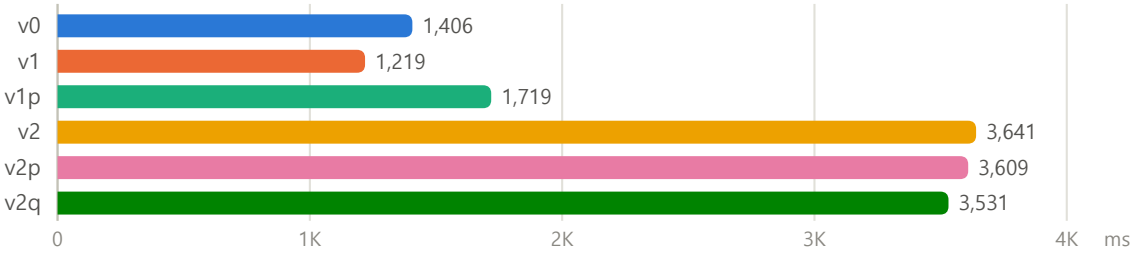
서버 uploadMs · events · postMs → uploadMs



서버 uploadMs · events · postConcurrentMs → uploadMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

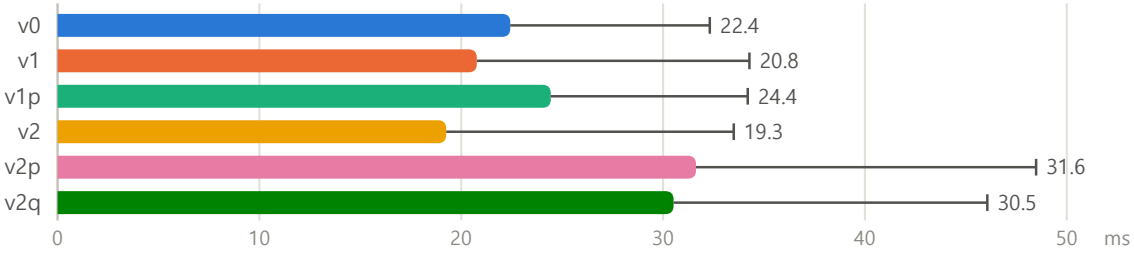
6. E2E 흐름 (mock) `e2e_mock`

POST → 10 ms 간격 폴링 → COMPLETED → GLB 다운로드(크기 확인), 순차. 워밍업 5 + 측정 50, 폴링 10 ms.

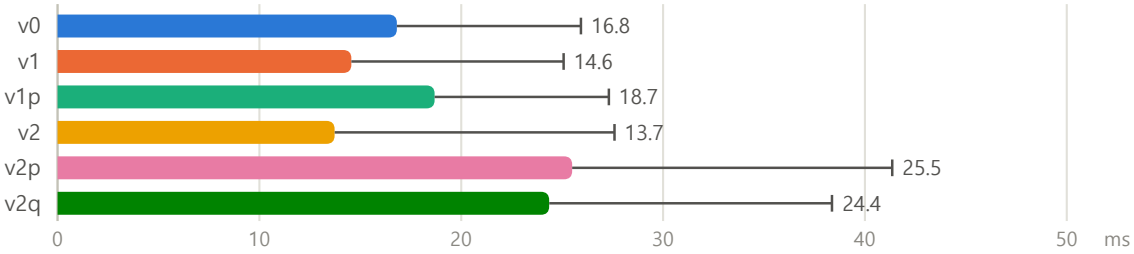
지표	v0	v1	v1p	v2	v2p	v2q
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	22.4 / 32.3 n=150 +8.0% = 차이 없음	20.8 / 34.3 n=150 기준	24.4 / 34.2 n=150 +17.6% = 차이 없음	19.3 / 33.5 n=150 −7.3% = 차이 없음	31.6 / 48.5 n=150 +52.3% ! 악화	30.5 / 46.1 n=150 +46.9% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	16.8 / 25.9 n=150 +15.5% = 차이 없음	14.6 / 25.1 n=150 기준	18.7 / 27.3 n=150 +28.3% = 차이 없음	13.7 / 27.6 n=150 −5.7% = 차이 없음	25.5 / 41.4 n=150 +75.2% ! 악화	24.4 / 38.4 n=150 +67.3% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	5.93 / 8.37 n=150 −0.7% = 차이 없음	5.97 / 10.4 n=150 기준	6.03 / 8.19 n=150 +0.9% = 차이 없음	5.62 / 8.47 n=150 −5.8% = 차이 없음	6.36 / 10.0 n=150 +6.5% = 차이 없음	6.41 / 10.7 n=150 +7.3% = 차이 없음
작업당 폴링 횟수 (평균) `bench.pollsPerJob` · 회 · 값	1.50 3회 +1.4%	1.48 3회 기준	1.54 3회 +4.0%	1.00 3회 −32.4%	1.64 3회 +10.8%	1.66 3회 +12.2%
서버 totalMs · events `flowMs → totalMs` · ms · p50 / p95	4.00 / 9.55 n=150 ±0.0% = 차이 없음	4.00 / 12.6 n=150 기준	4.00 / 8.00 n=150 ±0.0% = 차이 없음	3.00 / 5.55 n=150 −25.0% = 차이 없음	5.00 / 17.0 n=150 +25.0% = 차이 없음	6.50 / 16.0 n=150 +62.5% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	1,969 3회 +6.8%	1,844 3회 기준	2,141 3회 +16.1%	2,094 3회 +13.6%	3,156 3회 +71.2%	3,297 3회 +78.8%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.53 3회	0.58 3회	0.60 3회	0.48 3회	0.53 3회	1,001 3회

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선 · v2q 다음 버전 + 개선 2

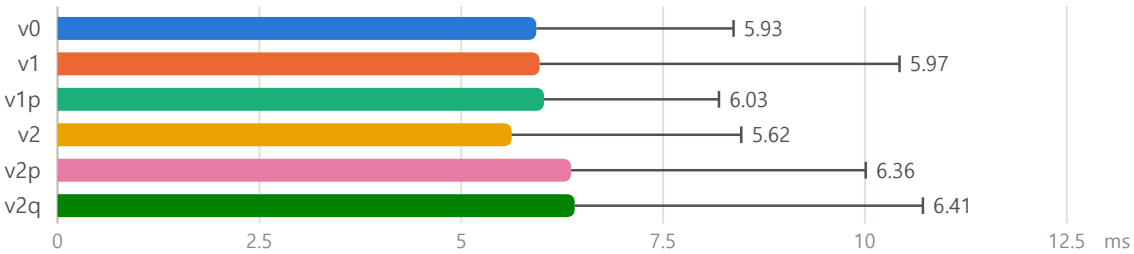
전체 흐름 (POST→GLB 수신) · flowMs



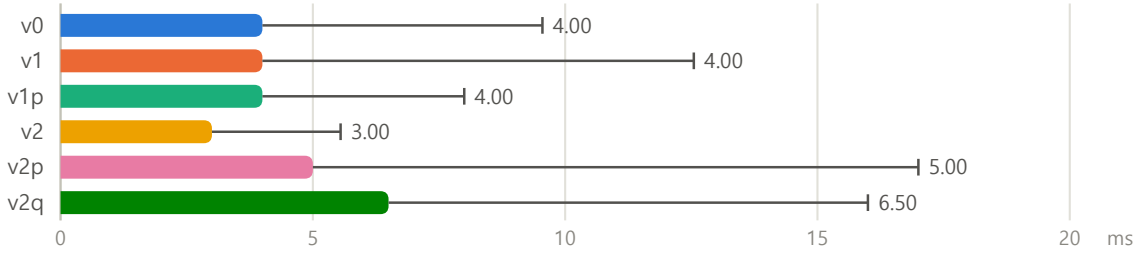
완료 관측 (POST→COMPLETED) · completeMs



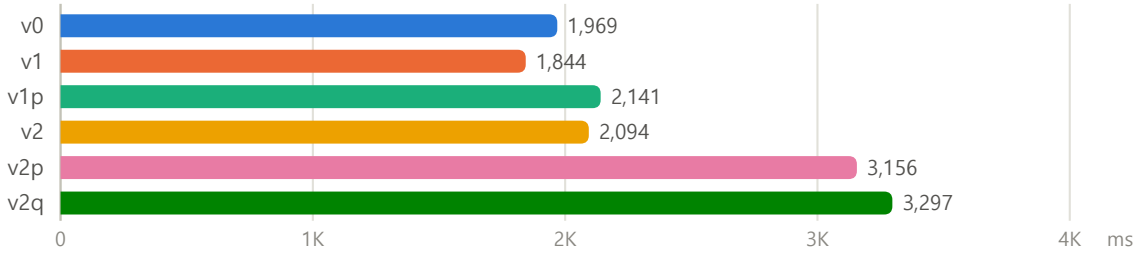
GLB 다운로드 · assetMs



서버 totalMs · events · flowMs → totalMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs

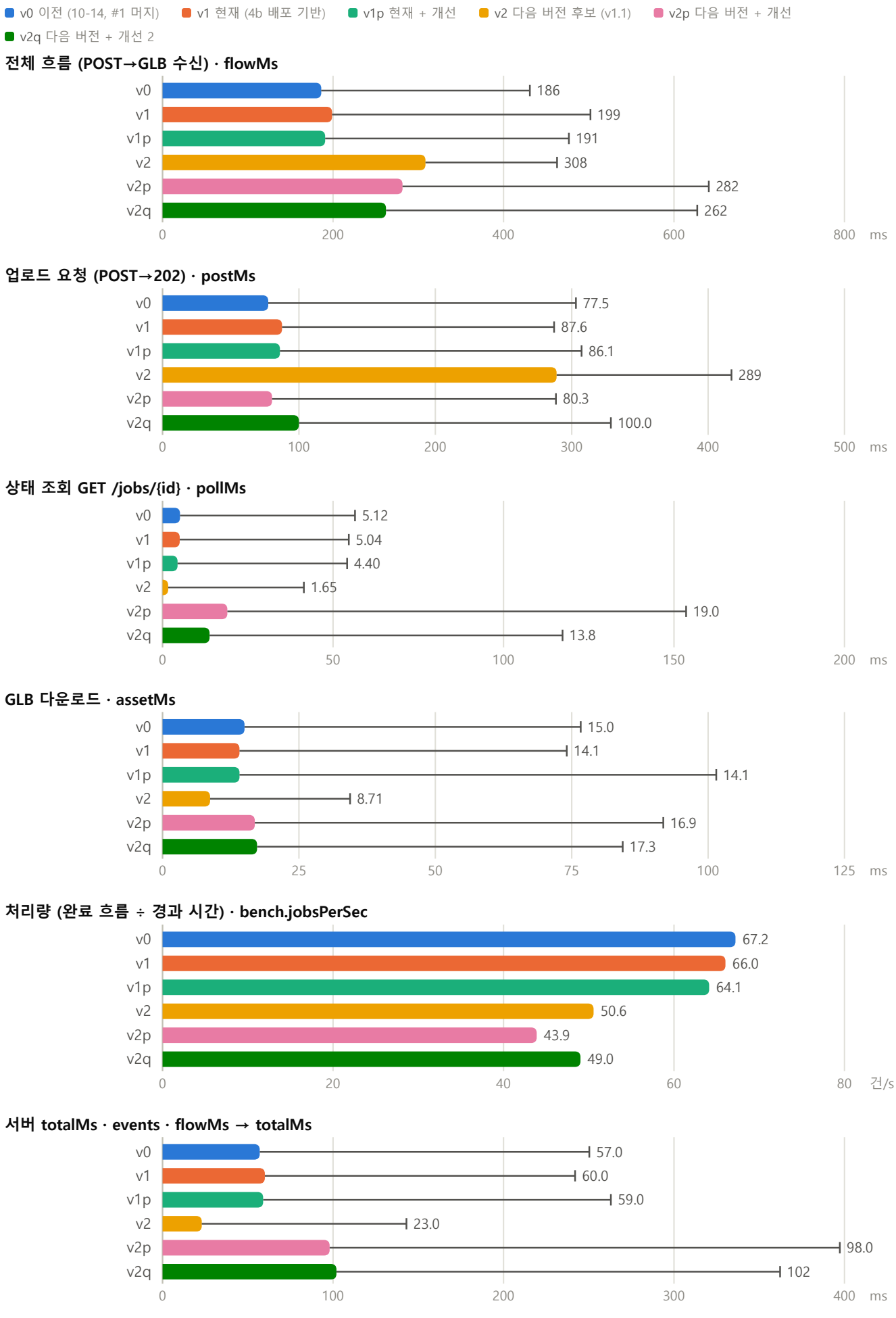


막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

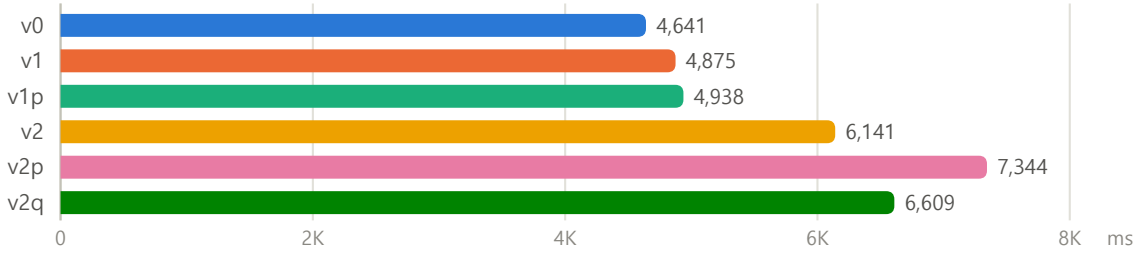
7. 동시 E2E 흐름 `concurrent_e2e`

16스레드가 각자 E2E 흐름을 반복, 폴링 간격 50 ms. 16스레드 × 10흐름, 폴링 50 ms.

지표	v0	v1	v1p	v2	v2p	v2q
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	186 / 431 n=480 −6.4% = 차이 없음	199 / 502 n=480 기준	191 / 477 n=480 −4.1% = 차이 없음	308 / 463 n=480 +55.1% ! 악화	282 / 641 n=480 +41.5% ! 악화	262 / 627 n=480 +31.8% ! 악화
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	77.5 / 303 n=480 −11.5% = 차이 없음	87.6 / 287 n=480 기준	86.1 / 307 n=480 −1.8% = 차이 없음	289 / 417 n=480 +229.8% ! 악화	80.3 / 288 n=480 −8.4% = 차이 없음	100.0 / 329 n=480 +14.1% = 차이 없음
상태 조회 GET /jobs/{id} `pollMs` · ms · p50 / p95	5.12 / 56.4 n=969 +1.7% = 차이 없음	5.04 / 54.6 n=1008 기준	4.40 / 54.1 n=979 −12.6% = 차이 없음	1.65 / 41.5 n=507 −67.2% ✓ 개선	19.0 / 154 n=1201 +277.4% ! 악화	13.8 / 117 n=1174 +173.9% ! 악화
GLB 다운로드 `assetMs` · ms · p50 / p95	15.0 / 76.7 n=480 +6.6% = 차이 없음	14.1 / 74.1 n=480 기준	14.1 / 101 n=480 ±0.0% = 차이 없음	8.71 / 34.4 n=480 −38.2% ✓ 개선	16.9 / 91.8 n=480 +19.9% ! 악화	17.3 / 84.4 n=480 +22.9% = 차이 없음
처리량 (완료 흐름 ÷ 경과 시간) `bench.jobsPerSec` · 건/s · 값	67.2 3회 +1.8% = 차이 없음	66.0 3회 기준	64.1 3회 −2.9% = 차이 없음	50.6 3회 −23.4% = 차이 없음	43.9 3회 −33.5% ! 악화	49.0 3회 −25.8% ! 악화
오류 (실패한 요청·흐름) `bench.errors` · 건 · 값	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계
서버 totalMs · events `flowMs → totalMs` · ms · p50 / p95	57.0 / 250 n=480 −5.0% = 차이 없음	60.0 / 242 n=480 기준	59.0 / 263 n=480 −1.7% = 차이 없음	23.0 / 143 n=480 −61.7% ✓ 개선	98.0 / 397 n=480 +63.3% ! 악화	102 / 362 n=480 +70.0% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,641 3회 −4.8%	4,875 3회 기준	4,938 3회 +1.3%	6,141 3회 +26.0%	7,344 3회 +50.6%	6,609 3회 +35.6%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.32 3회	0.30 3회	0.27 3회	0.28 3회	1,000 3회	1,000 3회


v0 이전 (10-14, #1 머지)
v1 현재 (4b 배포 기반)
v1p 현재 + 개선
v2 다음 버전 후보 (v1.1)
v2p 다음 버전 + 개선
v2q 다음 버전 + 개선 2
전체 흐름 (POST→GLB 수신) · flowMs
v0 186
v1 199
v1p 191
v2 308
v2p 282
v2q 262
0 200 400 600 800 ms
업로드 요청 (POST→202) · postMs
v0 77.5
v1 87.6
v1p 86.1
v2 289
v2p 80.3
v2q 100.0
0 100 200 300 400 500 ms
상태 조회 GET /jobs/{id} · pollMs
v0 5.12
v1 5.04
v1p 4.40
v2 1.65
v2p 19.0
v2q 13.8
0 50 100 150 200 ms
GLB 다운로드 · assetMs
v0 15.0
v1 14.1
v1p 14.1
v2 8.71
v2p 16.9
v2q 17.3
0 25 50 75 100 125 ms
처리량 (완료 흐름 ÷ 경과 시간) · bench.jobsPerSec
v0 67.2
v1 66.0
v1p 64.1
v2 50.6
v2p 43.9
v2q 49.0
0 20 40 60 80 건/s
서버 totalMs · events · flowMs → totalMs
v0 57.0
v1 60.0
v1p 59.0
v2 23.0
v2p 98.0
v2q 102
0 100 200 300 400 ms

서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

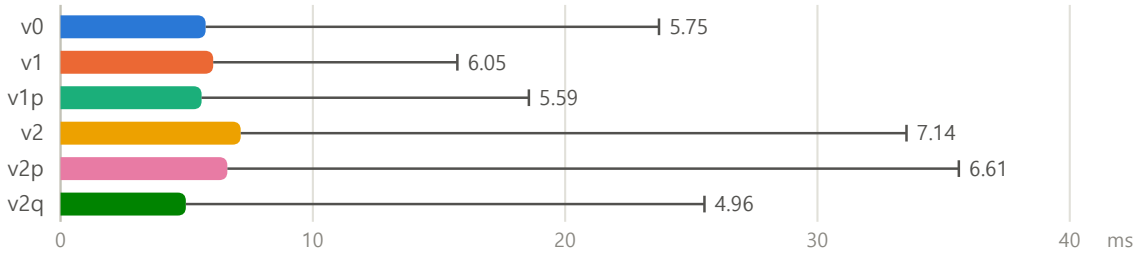
8. 폴링 부하 `poll_load`

COMPLETED 작업 하나를 4프로세스 × 4스레드가 GET /api/v1/jobs/{id} 로 반복 조회. 4프로세스 × 4스레드 × 250 (스레드마다 연결용 워밍업 5).

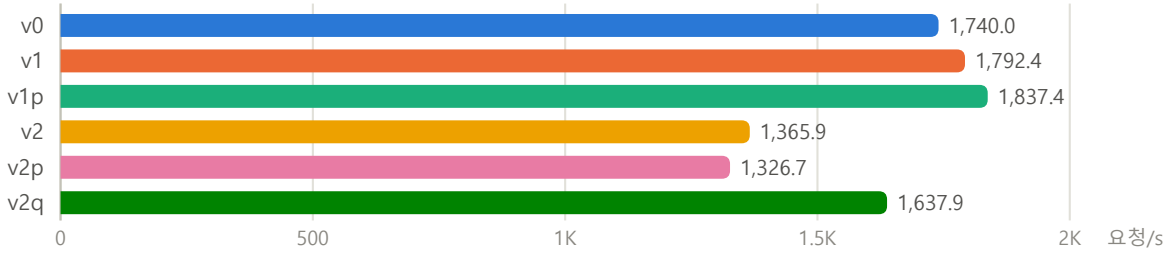
지표	v0	v1	v1p	v2	v2p	v2q
상태 조회 GET /jobs/{id} `pollMs` · ms · p50 / p95	5.75 / 23.7 n=12000 −5.0% = 차이 없음	6.05 / 15.7 n=12000 기준	5.59 / 18.6 n=12000 −7.6% = 차이 없음	7.14 / 33.5 n=12000 +18.0% = 차이 없음	6.61 / 35.6 n=12000 +9.3% = 차이 없음	4.96 / 25.5 n=12000 −17.9% = 차이 없음
처리량 (성공 요청 ÷ 경과 시간) `bench.pollReqPerSec` · 요청/s · 값	1,740.0 3회 −2.9% = 차이 없음	1,792.4 3회 기준	1,837.4 3회 +2.5% = 차이 없음	1,365.9 3회 −23.8% ! 악화	1,326.7 3회 −26.0% ! 악화	1,637.9 3회 −8.6% = 차이 없음
오류 (실패한 요청·흐름) `bench.errors` · 건 · 값	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,766 3회 +2.7%	4,641 3회 기준	4,547 3회 −2.0%	6,000 3회 +29.3%	6,234 3회 +34.3%	5,031 3회 +8.4%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	1,001 3회	1,001 3회	0.37 3회	0.24 3회	1,001 3회	1,001 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

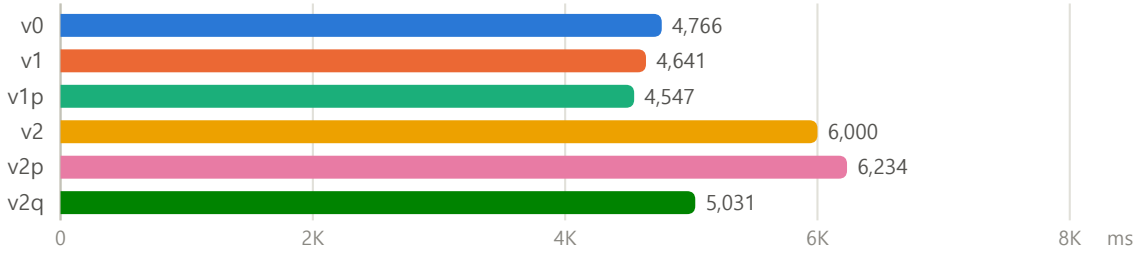
상태 조회 GET /jobs/{id} · pollMs



처리량 (성공 요청 ÷ 경과 시간) · bench.pollReqPerSec



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

9. 작업 수에 따른 변화 `scale`

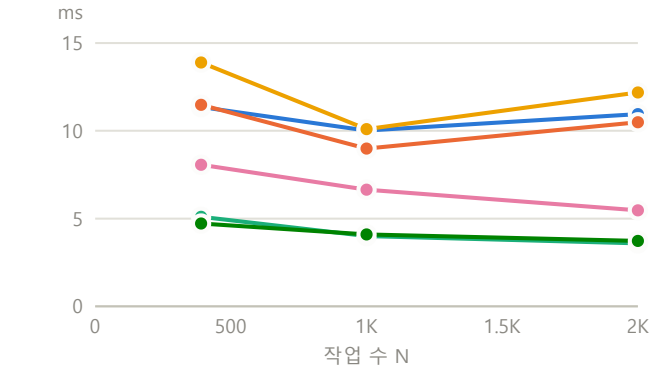
작업을 N 개까지 쌓은 뒤(8스레드 업로드) 목록 첫 페이지·중간 페이지, 단건 조회, 업로드. N = 200, 1000, 2000; 목록 첫 100, 중간 50, 조회 200, 업로드 30 (각 워밍업 5, 업로드 제외).

지표	v0	v1	v1p	v2	v2p	v2q
목록 첫 페이지 (limit 20) · N=390 (목표 200) `listFirstMs@200` · ms · p50 / p95	11.4 / 15.5 n=300 −1.1% = 차이 없음	11.5 / 17.0 n=300 기준	5.10 / 7.42 n=300 −55.6% ✓ 개선	13.9 / 18.0 n=300 +21.1% ! 악화	8.07 / 10.6 n=300 −29.7% ✓ 개선	4.72 / 7.60 n=300 −58.8% ✓ 개선
목록 첫 페이지 (limit 20) · N=1000 `listFirstMs@1000` · ms · p50 / p95	10.0 / 13.6 n=300 +11.5% = 차이 없음	8.98 / 12.4 n=300 기준	4.00 / 8.34 n=300 −55.5% ✓ 개선	10.1 / 15.6 n=300 +12.3% = 차이 없음	6.64 / 8.44 n=300 −26.0% = 차이 없음	4.10 / 6.32 n=300 −54.4% ✓ 개선
목록 첫 페이지 (limit 20) · N=2000 `listFirstMs@2000` · ms · p50 / p95	10.9 / 19.6 n=300 +4.4% = 차이 없음	10.5 / 18.6 n=300 기준	3.59 / 4.90 n=300 −65.8% ✓ 개선	12.2 / 19.4 n=300 +16.3% ! 악화	5.47 / 7.90 n=300 −47.8% ✓ 개선	3.73 / 5.11 n=300 −64.4% ✓ 개선
목록 중간 페이지 (limit 100) · N=390 (목표 200) `listDeepMs@200` · ms · p50 / p95	20.7 / 28.0 n=150 −1.0% = 차이 없음	20.9 / 26.5 n=150 기준	20.0 / 23.6 n=150 −4.3% = 차이 없음	35.9 / 41.5 n=150 +71.9% ! 악화	34.6 / 40.7 n=150 +65.7% ! 악화	19.3 / 25.6 n=150 −7.8% = 차이 없음
목록 중간 페이지 (limit 100) · N=1000 `listDeepMs@1000` · ms · p50 / p95	16.0 / 24.7 n=150 +9.8% = 차이 없음	14.5 / 24.0 n=150 기준	14.9 / 19.6 n=150 +2.2% = 차이 없음	25.5 / 39.5 n=150 +75.3% ! 악화	24.5 / 35.4 n=150 +68.7% ! 악화	15.2 / 18.2 n=150 +4.3% = 차이 없음
목록 중간 페이지 (limit 100) · N=2000 `listDeepMs@2000` · ms · p50 / p95	20.1 / 30.1 n=150 +8.9% = 차이 없음	18.5 / 29.0 n=150 기준	11.1 / 17.8 n=150 −40.1% ✓ 개선	28.3 / 34.0 n=150 +53.1% ! 악화	22.7 / 32.8 n=150 +23.0% = 차이 없음	12.1 / 17.5 n=150 −34.7% ✓ 개선
단건 조회 · N=390 (목표 200) `getMs@200` · ms · p50 / p95	0.53 / 0.79 n=600 +1.9% = 차이 없음	0.52 / 0.73 n=600 기준	0.54 / 0.74 n=600 +3.9% = 차이 없음	0.67 / 0.89 n=600 +28.9% ! 악화	0.74 / 1.04 n=600 +42.3% ! 악화	0.54 / 0.72 n=600 +3.9% = 차이 없음
단건 조회 · N=1000 `getMs@1000` · ms · p50 / p95	0.53 / 0.69 n=600 +10.4% ! 악화	0.48 / 0.61 n=600 기준	0.49 / 0.64 n=600 +2.1% = 차이 없음	0.63 / 0.81 n=600 +31.2% ! 악화	0.67 / 0.97 n=600 +39.6% ! 악화	0.50 / 0.69 n=600 +4.2% = 차이 없음
단건 조회 · N=2000 `getMs@2000` · ms · p50 / p95	0.46 / 0.61 n=600 ±0.0% = 차이 없음	0.46 / 0.60 n=600 기준	0.45 / 0.59 n=600 −2.2% = 차이 없음	0.56 / 0.75 n=600 +21.7% ! 악화	0.65 / 0.84 n=600 +41.3% ! 악화	0.50 / 0.69 n=600 +8.7% = 차이 없음
업로드 요청 (POST→202) · N=390 (목표 200) `postMs@200` · ms · p50 / p95	6.60 / 12.5 n=90 +4.6% = 차이 없음	6.31 / 10.1 n=90 기준	5.49 / 12.3 n=90 −13.1% = 차이 없음	10.5 / 13.9 n=90 +67.1% ! 악화	7.29 / 14.4 n=90 +15.5% = 차이 없음	7.75 / 17.9 n=90 +22.8% ! 악화
업로드 요청 (POST→202) · N=1000 `postMs@1000` · ms · p50 / p95	6.38 / 9.70 n=90 +22.6% ! 악화	5.21 / 10.3 n=90 기준	5.84 / 9.23 n=90 +12.2% = 차이 없음	9.62 / 12.2 n=90 +84.5% ! 악화	6.46 / 13.2 n=90 +23.9% = 차이 없음	7.53 / 13.8 n=90 +44.5% ! 악화

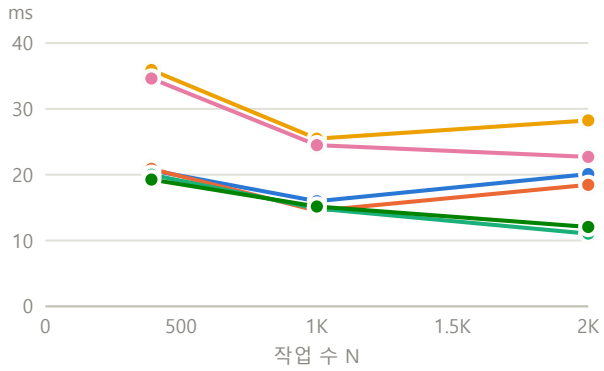
지표	v0	v1	v1p	v2	v2p	v2q
업로드 요청 (POST→202) · N=2000 `postMs@2000` · ms · p50 / p95	4.82 / 7.50 n=90 −1.6% = 차이 없음	4.90 / 7.13 n=90 기준	4.89 / 7.36 n=90 −0.1% = 차이 없음	9.35 / 11.9 n=90 +90.8% ! 악화	6.01 / 13.2 n=90 +22.8% ! 악화	6.01 / 12.4 n=90 +22.6% = 차이 없음
실제 작업 수 (목표 N=200) `bench.actualN@200` · 개 · 값	390 3회	390 3회	390 3회	390 3회	390 3회	390 3회
실제 작업 수 (목표 N=1000) `bench.actualN@1000` · 개 · 값	1,000 3회	1,000 3회	1,000 3회	1,000 3회	1,000 3회	1,000 3회
실제 작업 수 (목표 N=2000) `bench.actualN@2000` · 개 · 값	2,000 3회	2,000 3회	2,000 3회	2,000 3회	2,000 3회	2,000 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

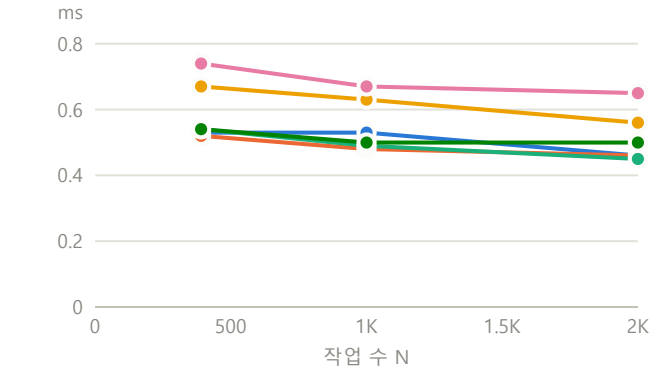
목록 첫 페이지 (limit 20) · listFirstMs p50



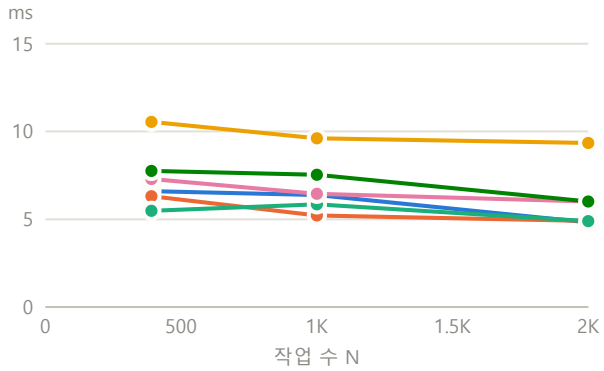
목록 중간 페이지 (limit 100) · listDeepMs p50



단건 조회 · getMs p50



업로드 요청 (POST→202) · postMs p50



가로축 = 측정 때 서버에 있던 작업 수(이전 시나리오가 만든 작업 포함), 세로축 = p50. 목록은 최신 작업부터, 중간 페이지는 서버가 준 nextCursor 로 약 N/2 위치에서 시작.

10. 자원 `resources`

mock 시나리오를 모두 마친 서버 A 의 최대 RSS(작업 집합 최대치)와 jar 크기. 1회.

지표	v0	v1	v1p	v2	v2p	v2q
최대 RSS `bench.peakRssMB` · MiB · 값	506.8 3회 −1.4% = 차이 없음	513.9 3회 기준	450.4 3회 −12.4% ✓ 개선	517.0 3회 +0.6% = 차이 없음	513.9 3회 ±0.0% = 차이 없음	509.2 3회 −0.9% = 차이 없음
jar 크기 `bench.jarBytes` · 값	52.9 MiB 3회 −0.1%	53.0 MiB 3회 기준	53.0 MiB 3회 ±0.0%	53.8 MiB 3회 +1.5%	53.8 MiB 3회 +1.5%	53.8 MiB 3회 +1.5%

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

최대 RSS · bench.peakRssMB



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

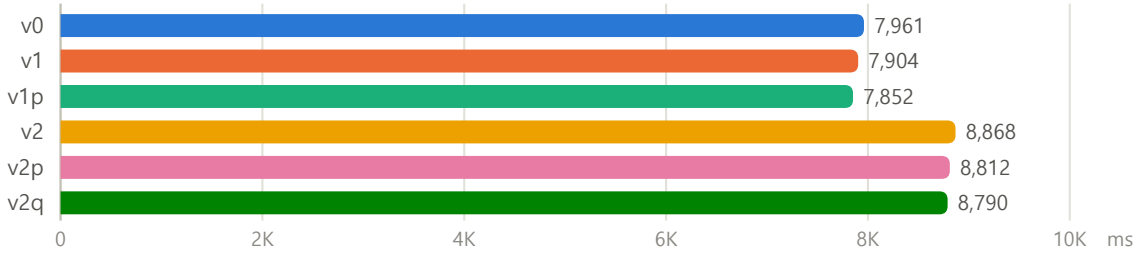
11. 재기동·PENDING 폴링 `pending_poll`

서버 A 의 저장소로 재기동(서버 B, 작업이 PENDING 에 머묾): 기동 시간, PENDING 작업 조회. PENDING 작업 5개, 워밍업 20 + 측정 200.

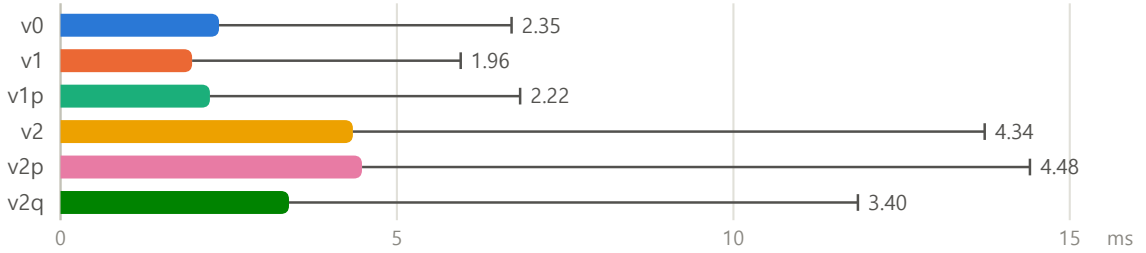
지표	v0	v1	v1p	v2	v2p	v2q
재기동 (Popen→healthz 200) `bench.restartStartupMs` · ms · 값	7,961 3회 +0.7% = 차이 없음	7,904 3회 기준	7,852 3회 −0.7% = 차이 없음	8,868 3회 +12.2% ! 악화	8,812 3회 +11.5% ! 악화	8,790 3회 +11.2% ! 악화
PENDING 작업 조회 `pendingPollMs` · ms · p50 / p95	2.35 / 6.70 n=600 +20.5% ! 악화	1.96 / 5.95 n=600 기준	2.22 / 6.83 n=600 +13.6% = 차이 없음	4.34 / 13.7 n=600 +122.2% ! 악화	4.48 / 14.4 n=600 +129.2% ! 악화	3.40 / 11.9 n=600 +73.7% ! 악화

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

재기동 (Popen→healthz 200) · bench.restartStartupMs



PENDING 작업 조회 · pendingPollMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

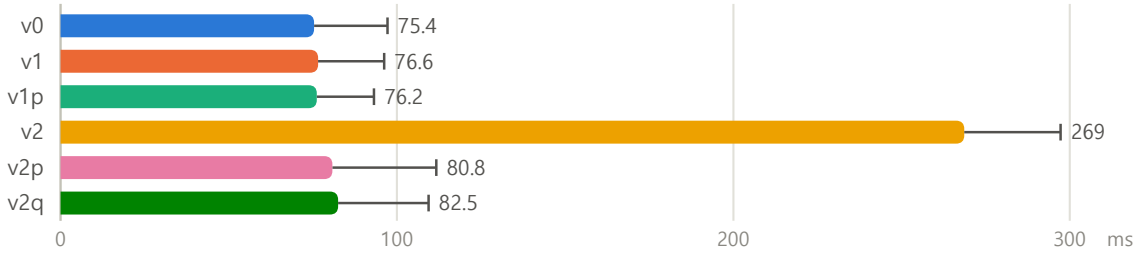
12. real · path 전송 `real_path`

real 프로파일 + 추론 스텁(패스스루, 모델 없음), 사진 경로 전달. 4.5 MB PNG 1장 E2E 순차, 이어서 5건 동시 접수. 워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건.

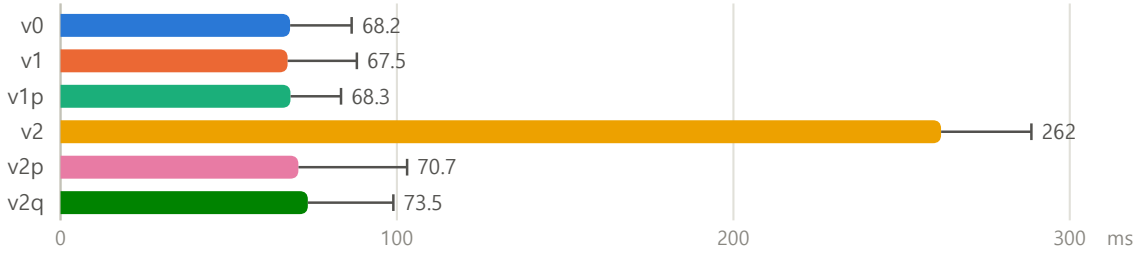
지표	v0	v1	v1p	v2	v2p	v2q
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	75.4 / 97.2 n=60 −1.6% = 차이 없음	76.6 / 96.2 n=60 기준	76.2 / 93.1 n=60 −0.5% = 차이 없음	269 / 297 n=60 +250.9% ! 악화	80.8 / 112 n=60 +5.5% = 차이 없음	82.5 / 109 n=60 +7.8% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	68.2 / 86.5 n=60 +1.1% = 차이 없음	67.5 / 88.1 n=60 기준	68.3 / 83.4 n=60 +1.3% = 차이 없음	262 / 289 n=60 +287.8% ! 악화	70.7 / 103 n=60 +4.8% = 차이 없음	73.5 / 98.9 n=60 +8.9% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	7.05 / 10.8 n=60 −13.5% = 차이 없음	8.15 / 11.7 n=60 기준	7.50 / 12.1 n=60 −8.0% = 차이 없음	7.73 / 12.2 n=60 −5.2% = 차이 없음	7.59 / 12.6 n=60 −6.8% = 차이 없음	8.21 / 13.5 n=60 +0.7% = 차이 없음
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	40.7 / 52.7 n=60 +2.4% = 차이 없음	39.8 / 55.7 n=60 기준	40.3 / 51.5 n=60 +1.4% = 차이 없음	257 / 285 n=60 +545.8% ! 악화	38.4 / 54.6 n=60 −3.4% = 차이 없음	41.2 / 53.9 n=60 +3.4% = 차이 없음
서버 queuedMs `serverQueuedMs` · ms · p50 / p95	11.5 / 24.1 n=60 −4.2% = 차이 없음	12.0 / 21.5 n=60 기준	12.0 / 24.0 n=60 ±0.0% = 차이 없음	10.0 / 16.0 n=60 −16.7% ✓ 개선	13.0 / 27.1 n=60 +8.3% = 차이 없음	14.0 / 26.1 n=60 +16.7% = 차이 없음
서버 processingMs `serverProcessingMs` · ms · p50 / p95	12.5 / 25.1 n=60 +13.6% = 차이 없음	11.0 / 20.0 n=60 기준	11.0 / 16.1 n=60 ±0.0% = 차이 없음	8.00 / 14.1 n=60 −27.3% ✓ 개선	15.0 / 29.1 n=60 +36.4% ! 악화	14.0 / 23.1 n=60 +27.3% ! 악화
서버 totalMs `serverTotalMs` · ms · p50 / p95	25.5 / 39.0 n=60 +6.2% = 차이 없음	24.0 / 40.2 n=60 기준	24.0 / 38.0 n=60 ±0.0% = 차이 없음	19.0 / 28.0 n=60 −20.8% ✓ 개선	28.5 / 43.2 n=60 +18.8% = 차이 없음	29.0 / 43.1 n=60 +20.8% = 차이 없음
작업 사이 빈 시간 `queueGapMs` · ms · p50 / p95	7.29 / 25.7 n=12 +71.8% = 차이 없음	4.24 / 14.2 n=12 기준	5.90 / 13.2 n=12 +39.1% = 차이 없음	5.07 / 17.5 n=12 +19.5% = 차이 없음	6.04 / 16.1 n=12 +42.3% = 차이 없음	9.19 / 35.8 n=12 +116.6% = 차이 없음
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	7,761 3회 −4.6% = 차이 없음	8,135 3회 기준	8,065 3회 −0.8% = 차이 없음	8,920 3회 +9.7% = 차이 없음	9,546 3회 +17.4% ! 악화	9,141 3회 +12.4% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,906 3회 +19.9%	4,094 3회 기준	3,875 3회 −5.3%	5,109 3회 +24.8%	5,859 3회 +43.1%	6,484 3회 +58.4%
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	3,010 3회	2,510 3회	2,009 3회	2,510 3회	5,512 3회	6,513 3회

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선 · v2q 다음 버전 + 개선 2

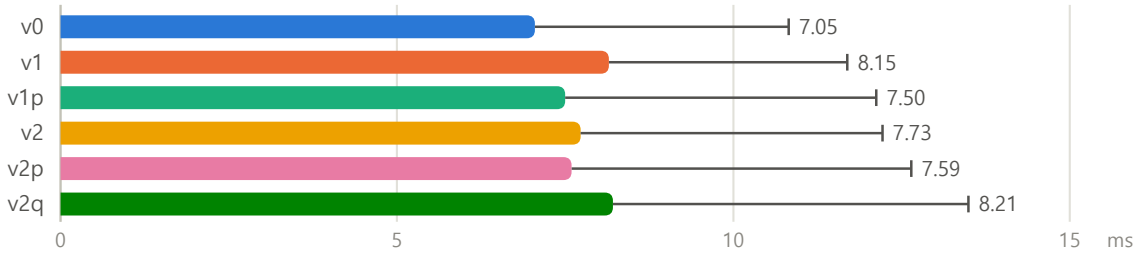
전체 흐름 (POST→GLB 수신) · flowMs



완료 관측 (POST→COMPLETED) · completeMs



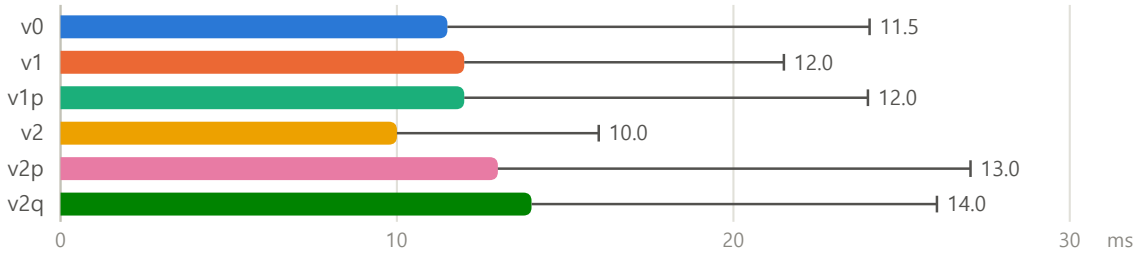
GLB 다운로드 · assetMs



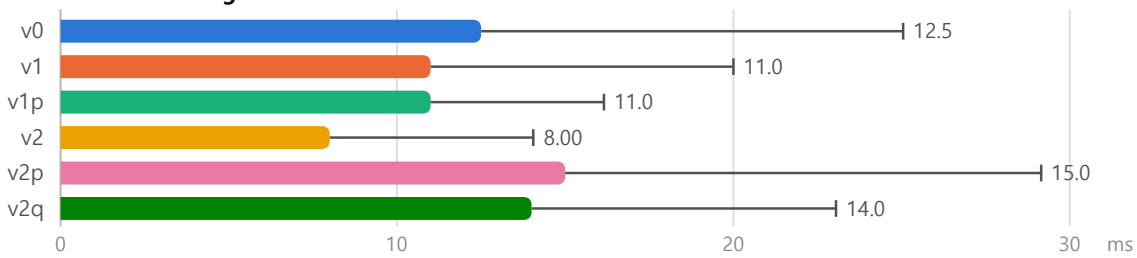
업로드 요청 (POST→202) · postMs



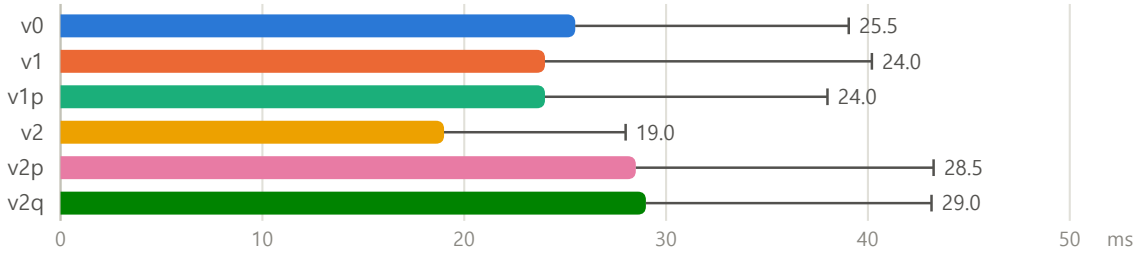
서버 queuedMs · serverQueuedMs



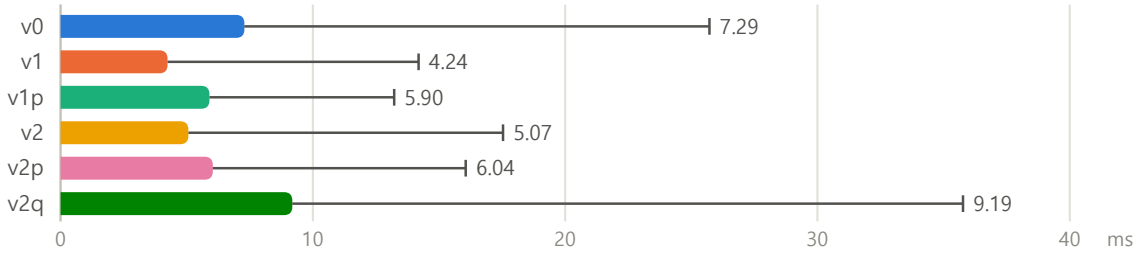
서버 processingMs · serverProcessingMs



서버 totalMs · serverTotalMs



작업 사이 빈 시간 · queueGapMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

13. real · upload 전송 `real_upload`

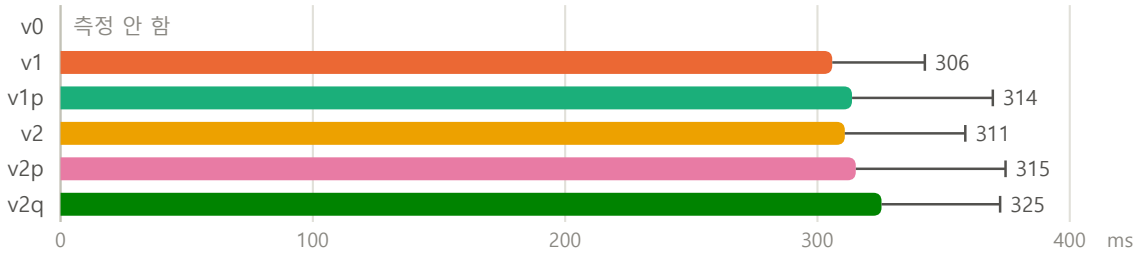
real_path 와 같고 /infer/upload 로 사진을 올리고 GLB 를 내려받는다. 워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건.

v0: 측정 안 함 — 이 버전에 없는 기능 (real_upload)

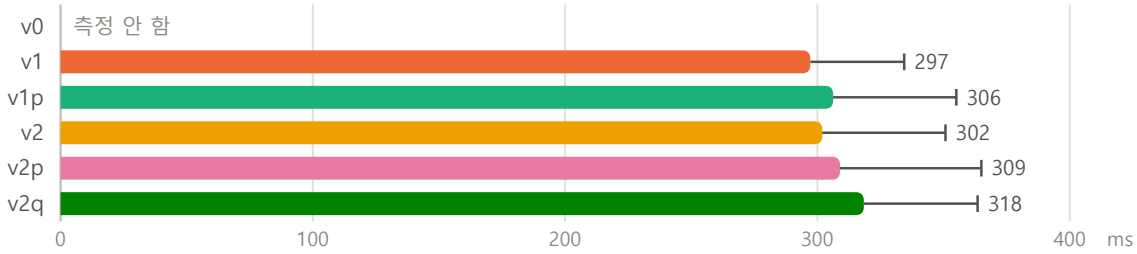
지표	v0	v1	v1p	v2	v2p	v2q
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	306 / 343 n=60 기준	314 / 369 n=60 +2.6% = 차이 없음	311 / 359 n=60 +1.6% = 차이 없음	315 / 375 n=60 +3.0% = 차이 없음	325 / 372 n=60 +6.4% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	297 / 334 n=60 기준	306 / 355 n=60 +3.0% = 차이 없음	302 / 351 n=60 +1.6% = 차이 없음	309 / 365 n=60 +4.0% = 차이 없음	318 / 363 n=60 +7.1% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	8.04 / 10.5 n=60 기준	7.80 / 10.5 n=60 −2.9% = 차이 없음	7.70 / 15.1 n=60 −4.2% = 차이 없음	7.04 / 12.0 n=60 −12.3% = 차이 없음	8.21 / 15.3 n=60 +2.1% = 차이 없음
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	39.2 / 53.6 n=60 기준	39.2 / 58.7 n=60 +0.1% = 차이 없음	266 / 290 n=60 +580.1% ! 악화	39.7 / 53.5 n=60 +1.2% = 차이 없음	41.9 / 55.2 n=60 +6.9% = 차이 없음
서버 queuedMs `serverQueuedMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	12.0 / 24.2 n=60 기준	10.0 / 21.1 n=60 −16.7% ✓ 개선	10.5 / 19.0 n=60 −12.5% = 차이 없음	13.5 / 29.1 n=60 +12.5% = 차이 없음	13.0 / 32.0 n=60 +8.3% = 차이 없음
서버 processingMs `serverProcessingMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	245 / 267 n=60 기준	248 / 298 n=60 +1.2% = 차이 없음	250 / 288 n=60 +1.8% = 차이 없음	250 / 292 n=60 +1.8% = 차이 없음	258 / 301 n=60 +5.1% = 차이 없음
서버 totalMs `serverTotalMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	260 / 283 n=60 기준	262 / 316 n=60 +0.8% = 차이 없음	264 / 300 n=60 +1.7% = 차이 없음	266 / 310 n=60 +2.5% = 차이 없음	270 / 312 n=60 +4.2% = 차이 없음
작업 사이 빈 시간 `queueGapMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	12.3 / 18.2 n=12 기준	12.1 / 20.1 n=12 −1.4% = 차이 없음	10.9 / 16.5 n=12 −11.3% = 차이 없음	11.7 / 23.0 n=12 −4.8% = 차이 없음	11.9 / 23.7 n=12 −3.0% = 차이 없음
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload)	8,018 3회 기준	8,077 3회 +0.7% = 차이 없음	9,102 3회 +13.5% ! 악화	8,820 3회 +10.0% ! 악화	9,065 3회 +13.1% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload)	7,219 3회 기준	7,219 3회 ±0.0%	6,625 3회 −8.2%	9,422 3회 +30.5%	10,656 3회 +47.6%
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload)	3,010 3회	2,511 3회	2,511 3회	3,510 3회	3,512 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

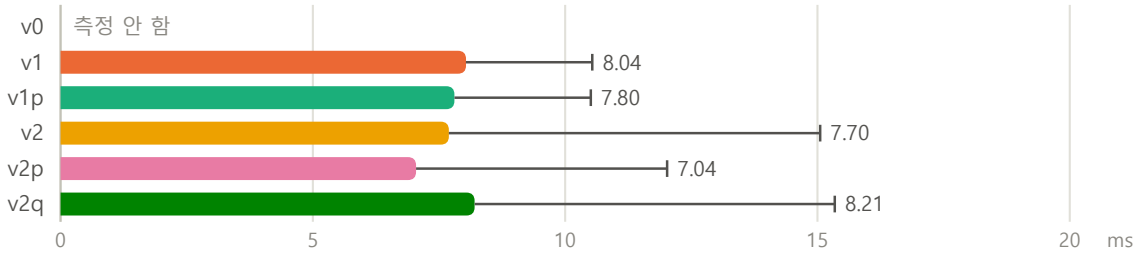
전체 흐름 (POST→GLB 수신) · flowMs



완료 관측 (POST→COMPLETED) · completeMs



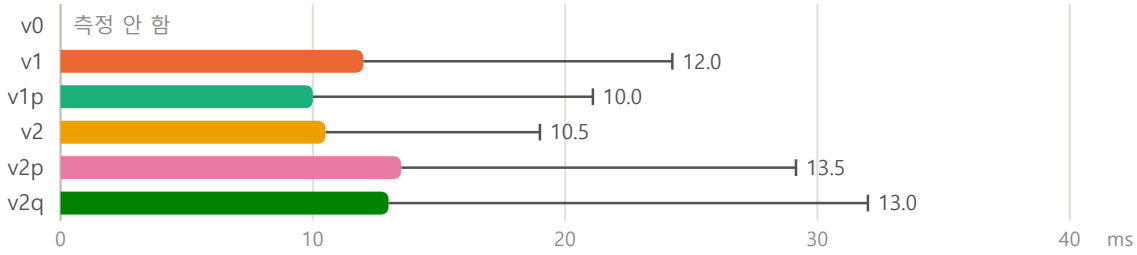
GLB 다운로드 · assetMs



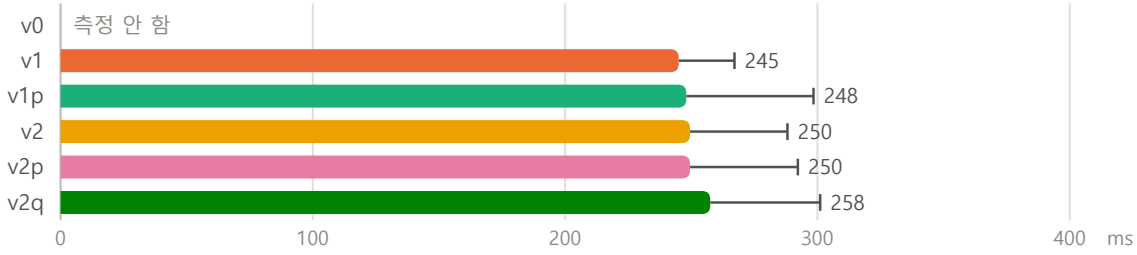
업로드 요청 (POST→202) · postMs



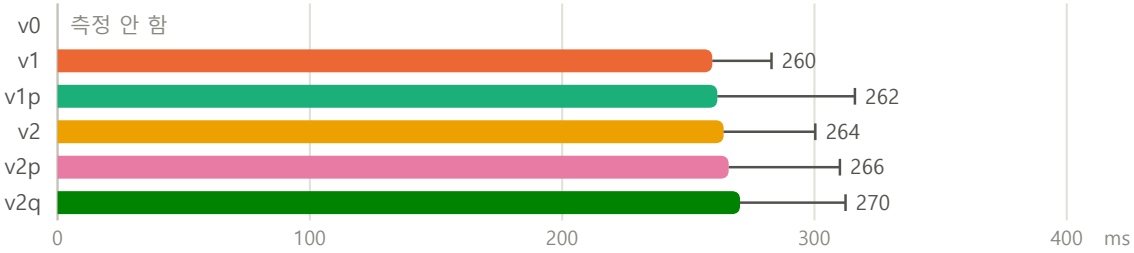
서버 queuedMs · serverQueuedMs



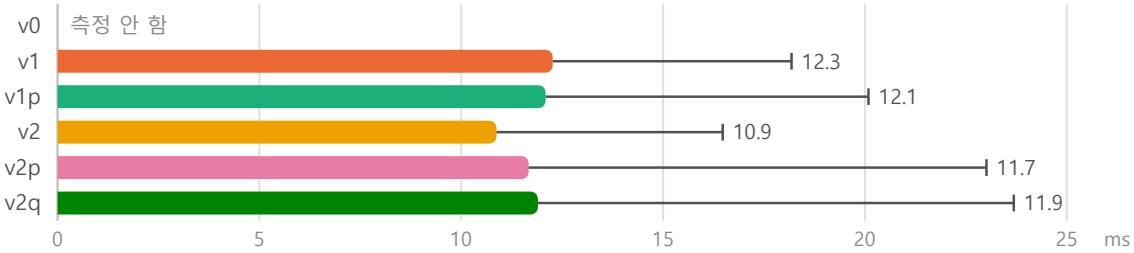
서버 processingMs · serverProcessingMs



서버 totalMs · serverTotalMs



작업 사이 빈 시간 · queueGapMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

14. real,prod · upload 전송 `real_upload_prod`

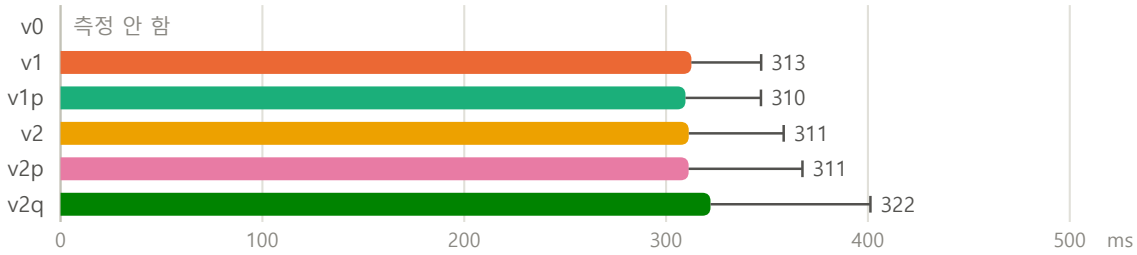
real,prod 프로파일: 사진 시그니처 검사, 작업마다 추론 서비스 healthz 확인. upload 전송. 워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건.

v0: 측정 안 함 — 이 버전에 없는 기능 (real_upload, prod)

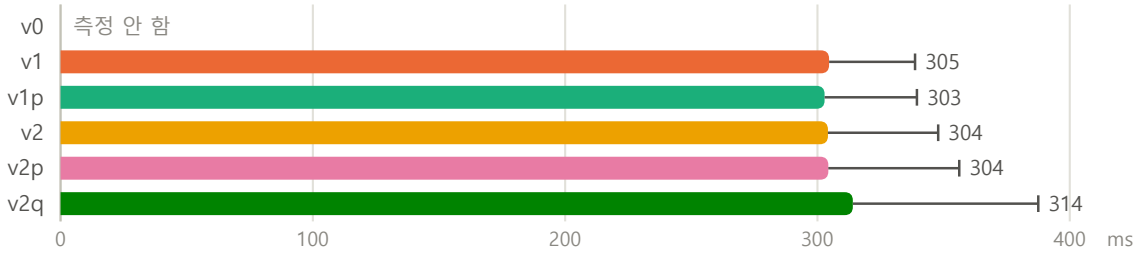
지표	v0	v1	v1p	v2	v2p	v2q
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	313 / 347 n=60 기준	310 / 347 n=60 −0.9% = 차이 없음	311 / 358 n=60 −0.4% = 차이 없음	311 / 368 n=60 −0.4% = 차이 없음	322 / 401 n=60 +3.0% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	305 / 339 n=60 기준	303 / 339 n=60 −0.6% = 차이 없음	304 / 348 n=60 −0.1% = 차이 없음	304 / 356 n=60 −0.1% = 차이 없음	314 / 388 n=60 +3.1% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	7.74 / 11.5 n=60 기준	6.71 / 10.9 n=60 −13.2% ✓ 개선	7.47 / 11.7 n=60 −3.4% = 차이 없음	7.12 / 9.80 n=60 −8.0% = 차이 없음	7.38 / 12.3 n=60 −4.5% = 차이 없음
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	37.7 / 49.2 n=60 기준	37.1 / 50.2 n=60 −1.6% = 차이 없음	267 / 314 n=60 +607.2% ! 악화	39.3 / 52.4 n=60 +4.3% = 차이 없음	38.4 / 57.4 n=60 +1.8% = 차이 없음
서버 queuedMs `serverQueuedMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	17.0 / 29.1 n=60 기준	15.5 / 24.1 n=60 −8.8% = 차이 없음	15.0 / 22.1 n=60 −11.8% = 차이 없음	25.0 / 41.0 n=60 +47.1% ! 악화	20.0 / 40.0 n=60 +17.6% = 차이 없음
서버 processingMs `serverProcessingMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	246 / 267 n=60 기준	242 / 278 n=60 −1.4% = 차이 없음	244 / 286 n=60 −0.6% = 차이 없음	240 / 290 n=60 −2.2% = 차이 없음	248 / 312 n=60 +1.2% = 차이 없음
서버 totalMs `serverTotalMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	266 / 285 n=60 기준	262 / 294 n=60 −1.1% = 차이 없음	260 / 306 n=60 −2.3% = 차이 없음	261 / 308 n=60 −1.7% = 차이 없음	273 / 350 n=60 +2.8% = 차이 없음
작업 사이 빈 시간 `queueGapMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	14.0 / 26.3 n=12 기준	14.8 / 18.8 n=12 +5.4% = 차이 없음	15.1 / 24.6 n=12 +7.4% = 차이 없음	19.5 / 33.0 n=12 +39.0% = 차이 없음	27.6 / 40.0 n=12 +96.6% ! 악화
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	7,658 3회 기준	8,008 3회 +4.6% = 차이 없음	8,681 3회 +13.4% ! 악화	8,462 3회 +10.5% ! 악화	8,563 3회 +11.8% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	7,141 3회 기준	7,062 3회 −1.1%	6,625 3회 −7.2%	9,875 3회 +38.3%	9,750 3회 +36.5%
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	2,509 3회	1,509 3회	2,510 3회	4,010 3회	2,511 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선 v2q 다음 버전 + 개선 2

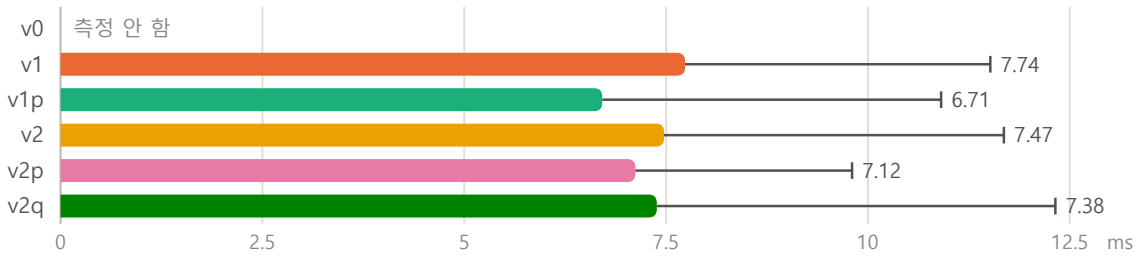
전체 흐름 (POST→GLB 수신) · flowMs



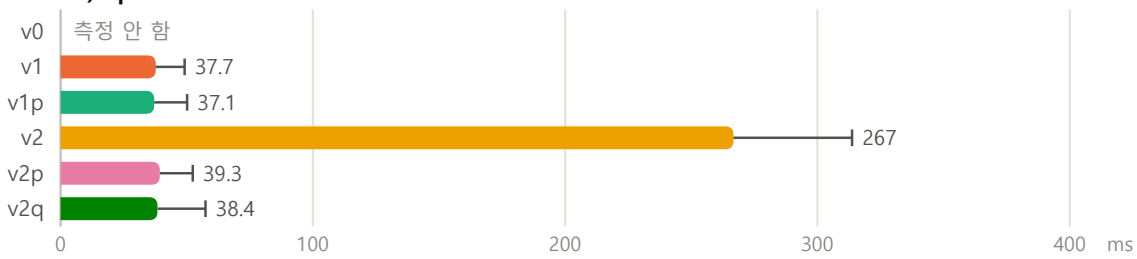
완료 관측 (POST→COMPLETED) · completeMs



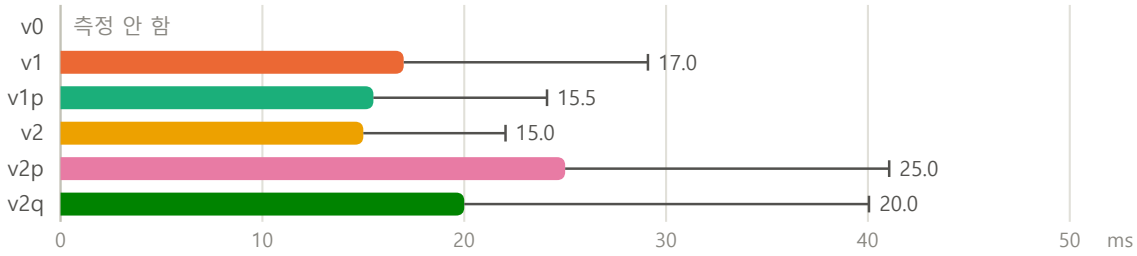
GLB 다운로드 · assetMs



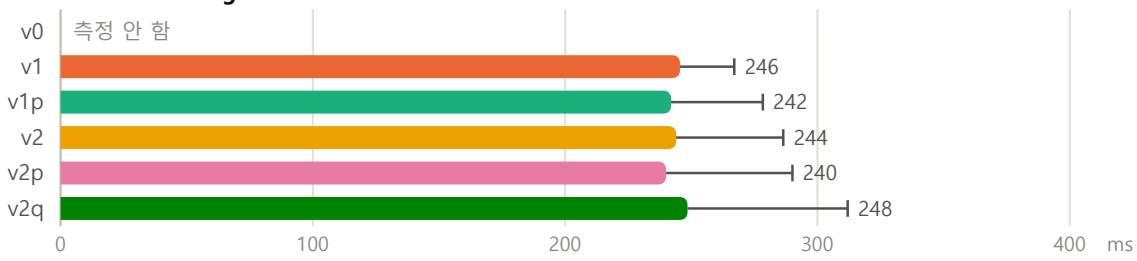
업로드 요청 (POST→202) · postMs



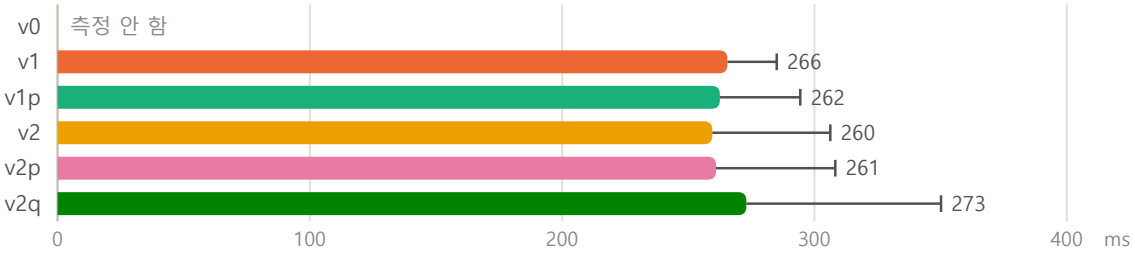
서버 queuedMs · serverQueuedMs



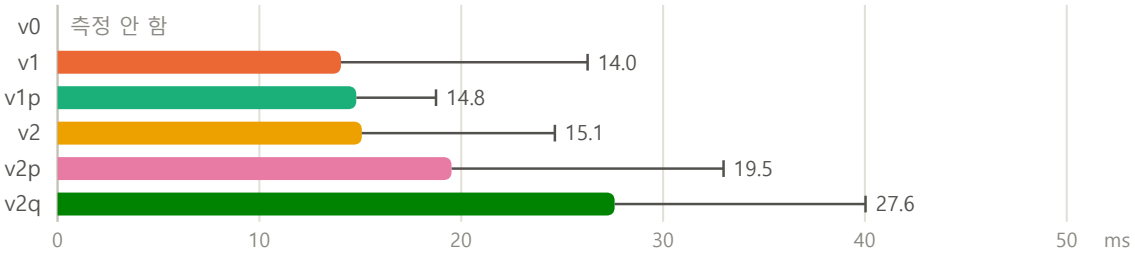
서버 processingMs · serverProcessingMs



서버 totalMs · serverTotalMs



작업 사이 빈 시간 · queueGapMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

개선 사례와 버전 비교

이 장의 숫자는 **[실험 결과]** 다. 표와 그림, P1~P5 절의 숫자는 이 보고서의 데이터(2026-11-01 12:57~14:02, 버전 6개 × 3회 교차, 같은 PC)에서 나왔다. P6·P7 의 원인 분리는 같은 날 바로 앞의 A/B 측정(`data/2026-11-01_1239_ablation`, 12:39~12:57, 버전 5개 × 3회, mock 시나리오 다섯 개만)에서 나왔다. 두 실행의 숫자는 섞어 판정하지 않는다. p50 은 세 반복을 합친 표본의 중앙값이다. 판정은 앞 장과 같은 규칙(|Δ| ≥ 10 % 이고 반복별 범위가 겹치지 않을 때만)이다. 원인 설명 가운데 확인하지 않은 것은 '가설'로 적었다. 하루 앞의 실행(`data/2026-10-31_1434`, v0~v2p)도 같은 결론을 냈다.

한눈에

비교	지표	앞 → 뒤 (p50)	변화	판정
v1 → v1p (P1)	목록 첫 페이지, N=2000	10.5 → 3.6 ms	−66 %	개선
v1 → v1p (P1)	목록 중간 페이지, N=2000	18.5 → 11.1 ms	−40 %	개선
v1 → v1p (P1)	최대 RSS	514 → 450 MiB	−12 %	개선
v1 → v2 (기능 비용)	12 MP 사진 업로드	13.9 → 64.4 ms	+364 %	악화
v1 → v2 (기능 비용)	real·path 업로드 요청	40 → 257 ms	+546 %	악화
v1 → v2 (기능 비용)	폴링 처리량	1,792 → 1,366 요청/s	−24 %	악화
v2 → v2p (P4)	12 MP 사진 업로드	64.4 → 14.6 ms	−77 %	개선
v2 → v2p (P4)	real·path 전체 흐름	269 → 81 ms	−70 %	개선
v2 → v2p (P1)	목록 첫 페이지, N=2000	12.2 → 5.5 ms	−55 %	개선
v2 → v2p (P5)	PENDING 작업 조회	4.35 → 4.48 ms	+3 %	차이 없음
v2p → v2q (P6)	폴링 처리량	1,327 → 1,638 요청/s	+23 %	개선
v2p → v2q (P6)	목록 중간 페이지, N≈390	34.6 → 19.3 ms	−44 %	개선
v2p → v2q (P6)	단건 조회, N≈390	0.74 → 0.54 ms	−27 %	개선
v2p → v2q (P7)	PENDING 작업 조회	4.48 → 3.40 ms	−24 %	개선

P1 작업 목록 키셋 페이지 (v1 → v1p)

- **실패 사례**: `GET /api/v1/jobs` 는 작업이 쌓일수록 느려질 구조였다. 요청마다 모든 작업을 읽어 정렬하고, 커서는 목록을 처음부터 훑어 찾았다.
- **원인 가설**: 비용이 페이지 크기가 아니라 전체 작업 수(N)에 비례한다. 같은 이유로 목록을 부를 때마다 모든 Job 엔티티가 메모리에 올라온다.
- **개선**: `(createdAt desc, id desc)` 키셋 조회에 limit+1, `jobs(created_at, id)` 인덱스. 커서는 그대로 마지막 항목의 id(계약의 '불투명 문자열' 유지, `JobListPagingTest`).
- **비교 실험**: N≈390·1000·2000 에서 첫 페이지(limit 20) 각 100회, 중간 페이지(limit 100) 각 50회.
- **결과**: 첫 페이지 p50 이 N≈390 에서 11.5 → 5.1 ms(−56 %), N=1000 에서 9.0 → 4.0 ms(−55 %), N=2000 에서 10.5 → 3.6 ms(−66 %). 중간 페이지는 N=2000 에서 18.5 → 11.1 ms(−40 %), N≈390·1000 은 차이 없음. 최대 RSS 가 514 → 450 MiB(−12 %)로 준 것은 목록마다 전체 작업을 올리지 않게 된 결과로 본다(가설). 단건 조회·업로드는 차이 없음.

P4 썸네일을 202 뒤 백그라운드로 (v2 → v2p)

- **실패 사례**: 다음 버전 후보 v2 에서 업로드가 크게 느려졌다(v1 대비): 12 MP 사진 13.9 → 64.4 ms, 큰 사진 4장 124 → 341 ms, real 시나리오의 4.5 MB 사진 업로드 요청 40 → 257~267 ms, 12 MP 4스레드 동시 업로드 49 → 222 ms.
- **원인 가설**: 업로드 요청 안에서 첫 사진의 썸네일을 디코드했다. 디코드는 서버 전체에서 한 번에 하나라서 동시 업로드가 줄을 선다. 이 PC 에서는 실시간 백신이 갓 쓴 사진을 처음 열 때 검사해(4.5 MB 노이즈 PNG 첫 열기 206 ms, 다시 열면 1.4 ms) 그 대기도 요청에 들어간다.
- **개선**: 썸네일은 202 를 보낸 뒤 스레드 하나의 큐에서 만든다. `thumbnailUrl` 은 만들어지기 전까지 null(계약의 '없으면 null' 범위).
- **결과**: 12 MP 업로드 64.4 → 14.6 ms(−77 %), 동시 업로드 222 → 65 ms(−71 %), 큰 사진 341 → 127 ms(−63 %), real 업로드 요청 257~267 → 38~40 ms(−85 %), real·path 전체 흐름 269 → 81 ms(−70 %). v1 대비로는 모두 차이 없음.
- **같이 본 것**: 일은 사라진 것이 아니라 옮겨 갔다. 12 MP 시나리오의 서버 CPU 점유 시간은 v2 3.6 s, v2p 3.6 s 로 같다(v1 은 1.2 s). 썸네일 큐가 뒤 측정에 섞이지 않도록 시나리오마다 서버가 쉴 때까지 기다린다(측정 방법). 그래도 쉬지 않고 이어지는 요청에서는 백그라운드 디코드와 CPU 를 나눈다(아래 '남은 비용').

P5 대기 순번 인덱스 (v2 → v2p)

- **실패 사례**: v2 에서 PENDING 작업 조회가 v1 의 1.96 ms 에서 4.35 ms(+122 %)로 느려졌다(작업 약 2,030개가 쌓인 DB).
- **원인 가설**: 폴링마다 대기 순번을 세는 `countQueuedBefore` 가 작업 표 전체를 훑는다.
- **개선**: `jobs(status, queued_at, id)` 인덱스와 범위 조건, `job_runs(job_id)·(finished_at)·jobs(pet_id)` 인덱스.

- **결과**: 4.35 → 4.48 ms(+3 %, **차이 없음**). 가설이 맞지 않았다: N≈2,030 에서는 전체 훑기도 싸다. 인덱스는 N 이 커질 때를 위해 남긴다. 실제 원인은 P7 에서 찾았다.

P6 썸네일 유무를 메모리에서 (review/api-v1.1 bd06de6)

- **실패 사례**: v1 대비 v2p 에서 폴링 처리량 −26 %, 단건 조회 +40~42 %(N≈390·1000·2000), 목록 중간 페이지와 첫 페이지도 항목마다 약 0.1 ms 씩 느렸다.
- **원인 가설**: `JobResponse.thumbnailUrl` 이 응답마다, 목록이면 항목마다 `Files.isRegularFile(썸네일)` 을 부른다. v1 에는 없던 파일 시스템 호출이다.
- **확인**: 같은 PC 에서 파일 2,000개로 잰 호출 한 번의 시간(6회 반복): 있는 파일 97~100 µs, 없는 파일 104~106 µs, 결과 GLB 의 `File.length` 103 µs, 메모리 집합 0.5 µs. 항목당 증가와 같은 크기다. 실시간 백신의 파일 시스템 필터를 거치는 Windows 의 값이라 Linux VM 에서는 훨씬 작을 것으로 본다(가설, VM 측정은 [계획]).
- **개선**: 썸네일이 있는 jobId 집합을 메모리에 둔다. 시작할 때 폴더를 한 번 읽어 채우고 만들기·지우기로 유지한다. 파일을 내주는 `find()` 와 드문 retry 만 디스크를 보고 집합을 바로잡는다. 응답 값은 같다(있으면 URL, 없으면 null).
- **결과**(A/B, P6 만 더한 v2f1): 폴링 처리량 1,346 → 1,685 요청/s(+25 %), 단건 조회 N≈390·1000·2000 에서 0.71/0.65/0.53 → 0.54/0.50/0.43 ms(−19~−24 %), 목록 첫 페이지 −31~−47 %, 중간 페이지 −31~−45 %. 모두 개선이고, v1(1,725 요청/s, 0.55/0.49/0.44 ms) 과 같은 수준이다. PENDING 조회와 E2E 흐름은 차이 없음.

P7 PENDING 대기 순번 조회를 트랜잭션 하나로 (review/api-v1.1 668d7a3·87e3c75)

- **실패 사례**: PENDING 작업 조회가 v1 대비 +122~129 %. P5 인덱스로도 그대로였다.
- **원인 가설**(코드와 라이브러리 소스로 확인):
- PENDING 응답마다 저장소 호출 두 번(`findProcessingStartedAt`, `countQueuedBefore`)이 각자 세션과 커넥션을 연다(`spring.jpa.open-in-view=false`).
- Spring Data JPA 3.2.0 은 `@Query` 문자열을 호출마다 ANTLR 로 다시 파싱한다(`QueryEnhancerFactory.forQuery` → `HqlQueryParser`; 정렬이 없어도 `renderSortedQuery` 가 파싱한다).
- 앞 보고서의 '예상 시간 중앙값 질의(5초 캐시)' 는 틀렸다: 캐시는 60초이고, 측정 구간에서는 첫 워밍업 한 번만 돈다.
- pending_poll 은 재시작 직후의 서버에서 워밍업 20회 뒤 200회를 잰다. 반복을 네 토막으로 나눈 중앙값(앞 실행)이 v1 3.3 → 1.8 ms, v2p 7.2 → 3.0 ms 로 내려가, JIT 전 코드에서 차이가 커진다(첫 50회 +3.9 ms, 마지막 50회 +1.3 ms).
- **개선**: 같은 JPQL 두 개를 `EntityManager` 로, 한 읽기 전용 트랜잭션 안에서 실행한다(저장소 조각 `JobQueueQueries`). 대기 중 조회 한 번의 세션 2 → 1(`QueueSnapshotTest`), ANTLR 재파싱 없음. 두 조회의 순서와 READ COMMITTED 의미는 그대로라 `ahead` 값도 같다.
- **비교 실험**(A/B): P6 만 넣은 v2f1 → 트랜잭션만 더한 실험 커밋 v2tx(`outlook` 에 `@Transactional(readOnly)`, `@Query` 경로 그대로) → P7 의 v2q.
- **결과**:
- 트랜잭션만(v2f1 → v2tx): PENDING 3.37 → 3.78 ms(+12 %, 차이 없음). 세션·커넥션을 하나로 묶는 것만으로는 효과가 없었다(가설 반박). 오히려 목록 중간 페이지(N≈390 +17 %)와 지연 0 mock 흐름(+17 %)이 악화했다: `outlook` 전체에 트랜잭션을 걸면 끝난 작업의 응답에도 트랜잭션이 생긴다.
- `@Query` 경로를 뺀 것(v2tx → v2q): PENDING 3.78 → 2.81 ms(−26 %, 개선). P7 의 몫은 ANTLR 재파싱을 포함한 Spring Data 문자열 쿼리 경로에서 나왔다(확인).
- 합계: A/B 에서 v2r → v2q 4.10 → 2.81 ms(−31 %, 개선), 이 보고서의 실행에서 v2p → v2q 4.48 → 3.40 ms(−24 %, 개선).
- **남은 차이**: v1 대비 A/B 에서 +30 %(범위가 겹쳐 차이 없음), 이 실행에서 +74 %(악화). v1 은 대기 순번을 계산하지 않으므로 조회 두 개는 기능의 비용이다. 재기동 직후를 재는 시나리오라 실행마다 흔들림도 크다.

이전 → 현재 (v0 → v1)

4b 배포 기반(prod 프로파일, 업로드 전송, GPU 감시·오프라인 대기, Discord 알림, H2 백업)을 더했지만 측정한 경로의 회귀는 없다: 기동, 메모리, 업로드, 동시 흐름, 폴링, 목록, real·path 모두 '차이 없음'. PENDING 조회만 2.36 → 1.96 ms(−17 %, 개선)로 나왔는데 원인은 확인하지 않았다.

다음 버전에 남은 비용 (v1 → v2q) [실험 결과] → 다음 개선 [계획]

v2q 가 기준 v1 보다 '악화'로 판정된 지표 전부다(목록 첫 페이지 −54~−64 %·N=2000 중간 페이지 −35 % 는 개선, 나머지는 차이 없음). P6·P7 뒤로 폴링 처리량(−9 %)·단건 조회·목록 중간 페이지는 v1 과 차이 없다.

지표	v1 → v2q (p50)	변화	가설	다음 확인
PENDING 조회	1.96 → 3.40 ms	+74 %	대기 순번 계산의 조회(v1 에 없음) + 재기동 직후 JIT 전 측정	pending_poll 워밍업을 늘려 정상 상태를 따로 보기, 두 조회를 문장 하나로
작은 사진 업로드	12.8 → 17.6 ms	+38 %	쉬지 않고 이어지는 업로드에서 앞 요청의 202 뒤 썸네일 일(디코드·파일 쓰기)과 CPU 공유. 사진마다 쉬고 재는 큰 사진·12 MP 순차 업로드는 v1 수준	업로드 사이에 쉬고 다시 재기, 썸네일 일 줄이기(EXIF 읽기 생략·JPEG writer 재사용)

지표	v1 → v2q (p50)	변화	가설	다음 확인
업로드 (N≈390·1000)	6.3 → 7.8, 5.2 → 7.5 ms	+23 %, +45 %	위와 같음(시드한 작업들의 썸네일 꼬리)	위와 같음
12 MP 4스레드 동시 업로드	48.8 → 58.1 ms	+19 %	동시 요청 사이에 썸네일 디코드	위와 같음
동시 E2E 처리량 (흐름 p50)	66.0 → 49.0 건/s (199 → 262 ms)	−26 % (+32 %)	작업마다 썸네일 CPU(v1 에 없는 기능의 일) + 대기·처리 중 폴링의 조회	썸네일이 없는 사진으로 대조 측정
real 처리 시간·작업 사이 빈 시간	path processing 11 → 14 ms, prod 빈 시간 14 → 28 ms	+27 %, +97 %	백그라운드 썸네일과 CPU 공유	GPU 추론(수십 초)에 비해 작아 따로 하지 않음
재기동	7.9 → 8.8 s	+11 %	빈(bean)·엔티티·`@Query` 검증 증가	시작 로그의 구간 시간

원인을 찾은 방법과 한계

- 다섯 경로(COMPLETED·PENDING 조회, 목록, 작은 업로드, 202 뒤 백그라운드)를 v1·v2p·tip 코드로 대조했고, 다른 검토자가 주장마다 반박을 시도했다. 그 결과 '목록 항목마다 N+1 조회'(업로드 목록은 JSON 열 하나), '완료된 작업도 대기 순번 계산'(COMPLETED 는 바로 없음) 같은 가설은 버렸다.
- JFR 프로파일은 이 PC(JDK 17.0.20, Windows)에서 Java 실행 표본(jdk.ExecutionSample)이 거의 잡히지 않아(요청 스레드 CPU 수 초에 표본 0~수십 개) 시간 분해에 쓰지 못했다. 대신 의심 호출의 마이크로벤치와, 의심 부분만 바꾼 커밋의 A/B 측정으로 확인했다(`versions-ablation.json`).
- e2e_mock 의 p50 은 첫 폴링에 끝나는 흐름의 비율이 바뀌면 10 ms 폴링 간격만큼 뛰는 문턱 효과다(앞 실행에서 첫 폴링 완료 v1 61 %, v2 83 %, v2p 35 %). 작업당 폴링은 이 실행에서 v1 1.48 → v2p 1.64 → v2q 1.66회다.
- A/B 실행은 앞 시나리오(기동·업로드)를 건너뛰어 e2e_mock 이 식은 서버의 첫 측정이 되므로, 절대값(v1 33 ms)이 이 보고서(21 ms)보다 크다. 같은 실행 안에서만 비교한다. A/B 에서 v2f1 → v2q 의 scale 업로드가 +19~32 %(악화)로 나왔지만 업로드 경로는 바뀐 코드가 없고 반복 범위도 넓어(n=90) 잡음으로 본다. 이 보고서의 실행에서는 v2p → v2q 업로드가 모두 차이 없음이다.

측정에서 고친 것

- 대기 없이 잰 빠른 실행(커밋하지 않은 동작 확인)에서는 v2p 의 동시 E2E·폴링이 앞 시나리오의 썸네일 디코드와 CPU 를 다퉈 2~3배 느리게 나왔다. 시나리오마다 서버가 쉴 때까지 기다리고, 서버 CPU 점유 시간을 함께 남기도록 도구를 고친 뒤 쟀다.
- 같은 PC 의 다른 프로그램이 고정한 CPU 를 차지해 서버가 멈추는 일이 있어 서버·스텁·부하 생성기를 보통 이상 우선순위로 띄운다. 게임이 켜진 동안에는 재지 않았다.
- 이 실행의 스위트 직전 배경 CPU 는 4.7~52 %(중앙 9 %)다. 13:30~13:46 에 같은 PC 의 다른 작업이 겹쳐 v2p 2회차가 배경 CPU 52 % 에서 시작했고 그 기동 시간이 10.1 s(다른 반복 8.3~8.8 s)로 이상치다. 같은 시간대의 v1p 3회차 폴링 처리량(1,634, 다른 반복 1,837~1,859 요청/s)과 v2q 2회차 작은 업로드(21.1 ms, 다른 반복 14.8~15.7 ms)도 튄다. 판정 규칙은 반복 범위가 겹치면 '차이 없음'으로 두므로 이런 반복은 판정을 약하게 할 뿐 만들지는 않는다. 다시 잴지는 [계획].
- 앞 보고서에서 한 번 본 'COMPLETED 직후 에셋 409' 는 2026-10-31 14:34 실행과 이번 두 실행에서 다시 나오지 않았다. 원인 미상 **[계획]**.

하지 않은 개선과 이유

- 업로드 모드의 정리(`DELETE /artifacts`)를 추론 슬롯 밖으로(P2): 슬롯을 더 잡는 시간이 작업당 왕복 1회(루프백 수 ms, Tailscale 수십 ms)라 수십 초 걸리는 추론에 비해 작다. VM ↔ GPU PC 실측 뒤 다시 본다.
- 업로드 저장을 복사 대신 이동(P3): VM 의 Docker 에서는 Tomcat 임시 폴더와 저장소 볼륨이 다른 파일시스템이라 결국 복사다. 닫지 않던 입력 스트림은 코드 리뷰에서 고쳤다.
- 결과 GLB 크기를 완료 때 저장(`asset.bytes` 의 `File.length` 제거, 항목당 약 0.1 ms): v1 에도 있던 비용이라 이번 회귀의 원인이 아니다. 열 추가와 retry 처리가 필요해 **[계획]**.
- 썸네일 일 줄이기(EXIF 읽기 생략, JPEG writer 재사용), 생성 시 merge SELECT 없애기(`Persistable`), 완료마다 다시 읽는 중앙값을 증분으로: 효과를 재지 않았거나 회귀와 무관해 **[계획]**.
- GLB 의 ETag/304, 목록 기본 기간, 응답 압축: 상태 코드나 앱 동작이 바뀌어 계약 회의 안건이다.

다음 버전 측정 절차와 남은 측정

다음 버전(v3)을 만들었을 때

1. `docs/perf/versions.json` 의 `versions` 에 `v3` 를 더한다. `ref` 는 잴 커밋의 **SHA** 로 적는다(브랜치는 움직여서 커밋한 보고서를 다시 만들 수 없다). `capabilities` 에는 그 버전에 있는 기능(`mock`, `real_path`, `real_upload`, `prod`)만 넣는다.
2. `order` 끝에 `v3` 를 넣는다. 보고서의 색은 이 순서의 자리로 정해지므로 앞 라벨의 자리는 바꾸지 않는다.
3. `pairs` 에 직전 버전과의 쌍을 더한다. 예: `{"from": "v2q", "to": "v3", "title": "v1.2 의 변화"}`. 배포 버전이 바뀌면 `baseline` 도 옮긴다.
4. 기준 버전·직전 버전과 **같은 실행에서** 잰다. 날짜가 다른 실행의 결과를 섞어 판정하지 않는다: 다른 프로그램의 부하, 온도, OS·JDK 업데이트가 실행마다 달라 판정 규칙(반복 사이 범위)이 그 차이를 걸러 내지 못한다.

```
python scripts/perf/run.py --versions v1,v2q,v3 --reps 3 --versions-file docs/perf/versions.json --stub-python <fastapi·uvicorn 이 깔린 python>
python scripts/perf/report.py --data docs/perf/data/<새 폴더> --versions-file docs/perf/versions.json --notes docs/perf/notes --out docs/perf/backend-perf-report.pdf
```

5. 개선이 있으면 '개선 사례'와 같은 순서(실패 사례 → 원인 가설 → 개선 → 비교 실험)로 `docs/perf/notes/` 에 적는다. 새 데이터 폴더, PDF, 노트를 한 커밋에 올린다.
6. 원인이 여럿 겹쳐 있으면 의심 부분만 바꾼 커밋을 따로 만들어 `versions-ablation.json` 처럼 별도 버전 파일로 A/B 측정한다(P7 에서 트랜잭션 묶과 `@Query` 재파싱 몫을 이렇게 나눴다). 이 PC 에서는 JFR 실행 표본이 거의 잡히지 않아 프로파일러로는 나누지 못했다.

측정 전 점검

- 무거운 프로그램(게임 클라이언트, 화상 회의, 다른 빌드, 여러 에이전트를 띄우는 다른 세션)을 끈다. 그 부하는 보고서 '한계'에 스위트마다 잰 CPU 사용률로 적힌다.
- 측정 중에 PC 가 절전에 들어가지 않게 한다(2026-11-01 실행은 밤사이 시작이 8시간 밀렸다).
- 노트북이면 전원을 연결하고 절전 모드를 끈다.
- `--quick --reps 1` 로 한 번 돌려 모든 시나리오가 `ok` 인지 먼저 본다(약 1분 × 버전 수).

아직 재지 않은 것 [계획]

항목	왜 필요한가	방법	담당
클라우드 VM(2 vCPU)에서 같은 시나리오	이 보고서는 개발 PC 에서 서버를 논리 CPU 2개(물리 코어 1개)에 고정해 VM 을 흉내 낸 값이다	VM 에 JDK 17·Python 을 두고 같은 도구를 실행한다. 개발 PC 에서 VM 으로 원격 부하를 거는 것은 도구 확장이 필요하다	Backend
VM ↔ GPU PC(Tailscale) 업로드 모드의 전송 시간	사진 업로드, GLB 다운로드, `DELETE /artifacts` 가 루프백이 아니라 실제 망에서 얼마인지	VM 의 Spring(`real,prod`, 업로드 모드)과 GPU PC 의 추론 서비스로 E2E. events.jsonl 의 `processingMs` 와 `inferMs`·`convertMs` 의 차이가 전송·저장 시간이다	Backend + 3D
실제 AnimalLift 추론 시간과 GPU 메모리	이 보고서의 real 시나리오는 모델 없는 패스스루 스텁이라 추론 시간이 0 이다	exp-01 (`generation/experiments/`)	3D
Unity 로딩 시간과 FPS	서버 수치만으로는 사용자가 기다리는 시간을 다 설명하지 못한다	앱의 metrics.csv (METRICS §3·§4)	Unity

부록

데이터 파일

- `docs/perf/data/2026-11-01_1257`: run.json, v0-r1.json, v0-r2.json, v0-r3.json, v1-r1.json, v1-r2.json, v1-r3.json, v1p-r1.json, v1p-r2.json, v1p-r3.json, v2-r1.json, v2-r2.json, v2-r3.json, v2p-r1.json, v2p-r2.json, v2p-r3.json, v2q-r1.json, v2q-r2.json, v2q-r3.json

명령

- 측정: `python scripts/perf/run.py --versions v0,v1,v1p,v2,v2p,v2q --reps 3 --versions-file docs/perf/versions.json --stub-python C:/Users/minwo/AppData/Local/Temp/claude/C--Users-minwo-OneDrive-------capstone-Beside/7b66930a-7eb1-4f94-991f-a3a35ae2caf2/scratchpad/venv/Scripts/python.exe` (2026-11-01T12:57:46+09:00 ~ 2026-11-01T14:02:34+09:00, 상태 ok)
- 보고서: `python scripts/perf/report.py --data docs/perf/data/2026-11-01_1257 --versions-file docs/perf/versions.json --notes docs/perf/notes --out docs/perf/backend-perf-report.pdf`

도구와 버전 커밋

- 측정 도구 커밋 `ce80e472b0d0a065c5929e8b8e5a46bb9514cf32`
- 보고서 생성 도구 커밋 `4a8e6d26b0bbe05391e8defbe6acf4cf48471191`
- v0: `e0e1c2b34b1b94669d824fd0fd4afeea9fb800f1` (e0e1c2b34b1b94669d824fd0fd4afeea9fb800f1)
- v1: `af4a747b1c36f2380d1ff69a523b5a9dbaca7503` (af4a747b1c36f2380d1ff69a523b5a9dbaca7503)
- v1p: `033f3f3719745e3d4776c5ea6e4e2e7db536f027` (033f3f3719745e3d4776c5ea6e4e2e7db536f027)
- v2: `6a3959f60380947ad2f430be24f5d50661a57c95` (6a3959f60380947ad2f430be24f5d50661a57c95)
- v2p: `ecc5fb5f9ba0bee443137ce8c59a5223174e7dc2` (ecc5fb5f9ba0bee443137ce8c59a5223174e7dc2)
- v2q: `87e3c75ce1be65a4a7eacd692b4ab115298a422e` (87e3c75ce1be65a4a7eacd692b4ab115298a422e)

생성 2026-11-01T14:10:17+09:00